StockGuard AI

A Predictive Business Intelligence System for SMEs



Muhammad Ahsan Qasim (SP-2023-BSCS-041)
Arslan Ahmad (SP-2023-BSCS-043)

Supervised By
Miss Tayyaba Sultana

Submitted for the partial fulfillment of the BS Computer Science degree
to the Faculty of Computer Science

LAHORE GARRISON UNIVERSITY
October 2026

ABSTRACT

Small and medium-sized enterprises frequently manage inventory with spreadsheets, manual estimates and limited analytical support. These practices make it difficult to anticipate product-level demand, determine appropriate reorder quantities and distinguish temporary fluctuations from persistent change. The resulting stockouts, excess inventory and weak revenue planning can directly affect cash flow and customer service. StockGuard AI addresses this problem through a cross-platform predictive business intelligence system designed around the operational needs of SMEs.

The implemented solution combines a React web application, a React Native mobile application, a Node.js and Express business API, MongoDB Atlas and a Python forecasting service. Historical sales uploaded through validated CSV files are transformed into continuous daily time series. Linear Regression, ARIMA and Random Forest candidates are evaluated with MAE and RMSE, while an imported global XGBoost model and a seasonal-naive fallback provide additional production forecasting paths. Forecasts are available for 7, 14, 28 and 30 days and store every daily prediction, the selected model, accuracy metrics, total demand and estimated revenue.

StockGuard AI converts forecasts into inventory decisions by calculating lead-time demand, demand variability, safety stock, reorder points, target stock, days of cover and recommended order quantities. It generates low-stock, critical-stock, overstock and demand-spike recommendations and tracks each recommendation through open, approved, dismissed and completed states. The system also implements role-based access, tenant isolation, audit logs, in-app and push-notification pipelines, reports, encrypted daily backups and isolated restore verification.

The production XGBoost bundle was trained on 8,172,438 rows and evaluated through a three-fold recursive backtest containing 986,433 observations. It achieved MAE 0.574939 and RMSE 2.021802, compared with MAE 0.658729 and RMSE 2.459768 for the seasonal baseline, a 12.72 percent MAE improvement. A production load test with 100 authenticated virtual users returned 100 successful responses, an average of 259 ms and p95 of 832 ms. The completed prototype demonstrates a practical path from historical sales to a defensible replenishment decision in one secure web and mobile platform.

Keywords: Demand forecasting, inventory management, business intelligence, XGBoost, ARIMA, Random Forest, Linear Regression, SMEs, MERN, replenishment.

CERTIFICATE

It is certified that the project report titled “StockGuard AI: A Predictive Business Intelligence System for SMEs,” submitted by Muhammad Ahsan Qasim and Arslan Ahmad for partial fulfillment of the requirements of the Bachelor of Science in Computer Science, is presented for internal defence and evaluation.

The project has been completed under the supervision of Miss Tayyaba Sultana in the Department of Computer Science, Lahore Garrison University. Committee members may record their approval and date below.

Name / Designation	**Signature / Date**
Head of Department	_____
Final Year Project Coordinator	_____
Miss Tayyaba Sultana — Supervisor	_____

COPYRIGHTS

This report documents the original academic and implementation work completed for StockGuard AI by Muhammad Ahsan Qasim and Arslan Ahmad during their BS Computer Science programme at Lahore Garrison University.

No part of the project may be reproduced for academic credit by another party without appropriate acknowledgement. Third-party libraries, datasets and published methods remain subject to their respective licences and citations.

Name / Designation	**Signature / Date**
Muhammad Ahsan Qasim	_____
Arslan Ahmad	_____

DECLARATION

We declare that this dissertation and the accompanying software prototype are our own work except where sources, libraries and datasets are explicitly acknowledged. The reported implementation evidence and test results were produced from the StockGuard AI repository and its isolated production workspace.

We understand that plagiarism, fabricated evidence or unacknowledged reuse may result in academic penalties under university policy.

Name / Designation	**Signature / Date**
Muhammad Ahsan Qasim	_____
Arslan Ahmad	_____

PLAGIARISM CERTIFICATE

This is to certify that the report titled “StockGuard AI: A Predictive Business Intelligence System for SMEs” is submitted for the award of the Bachelor of Science in Computer Science and was prepared under academic supervision.

The official similarity report and any required departmental plagiarism documentation shall be attached by the students or department before final submission.

Name / Designation	**Signature / Date**
Miss Tayyaba Sultana — Supervisor	_____

ACKNOWLEDGMENT

We are grateful to our supervisor, Miss Tayyaba Sultana, for her guidance, careful review and continued encouragement throughout the design, development and verification of StockGuard AI. Her feedback helped us connect the technical implementation with the practical problems faced by small and medium-sized businesses.

We also thank the Department of Computer Science and the Final Year Project committee at Lahore Garrison University for providing the academic framework for this work. We acknowledge the open-source communities behind React, Node.js, MongoDB, Python, scikit-learn, Statsmodels, XGBoost, React Native and Expo, whose tools made the cross-platform prototype possible.

Finally, we thank our families, friends and classmates for their patience and support. Their encouragement was particularly valuable during data preparation, model training, deployment and the final verification phase.

Table of Contents

ABSTRACT 2
ACKNOWLEDGMENT 7
Table of Contents 8
List of Figures 17
List of Tables 18
INTRODUCTION 19
1.1 Background 19
1.2 Project Motivation 19
1.3 Problem Statement 19
1.4 Aim and Objectives 20
1.5 Research Questions 20
1.6 Significance of the Study 20
1.7 Scope and Delimitations 20
1.8 Structure of the Report 21
1.9 SME Inventory Management Context 21
1.10 Stakeholders and Expected Benefits 21
1.11 Operational Scenario 22
1.12 Success Criteria 23
1.13 Definitions and Abbreviations 23
1.14 Chapter Summary 23
LITERATURE REVIEW 24
2.1 Introduction 24
2.2 Linear Regression 24
2.3 ARIMA 24
2.4 Random Forest and Gradient-Boosted Trees 24
2.5 Model Evaluation 25
2.6 Forecast-Based Inventory Control 25
2.7 Business Intelligence and Visualisation 25
2.8 Security and Multi-Tenant SaaS 25
2.9 Research Gap 26
2.10 Chapter Summary 26
2.10 Characteristics of Retail Demand 26
2.11 Time-Series Preparation 27

2.12 Feature Engineering for Demand Forecasting 27
2.13 Forecast Evaluation and Selection 28
2.14 Inventory Control Concepts 28
2.15 Business Intelligence for SMEs 29
2.16 Software Architecture and Cross-Platform Delivery 29
2.17 Security and Privacy Literature Applied to the Project 30
2.18 Comparative Synthesis 30
2.19 Refined Research Gap 30
2.20 Chapter Summary 31
PROBLEM STATEMENT AND PROPOSED FRAMEWORK 32
3.1 Formal Problem Definition 32
3.2 Stakeholders and User Classes 32
3.3 Delivered Functional Areas 32
3.4 Proposed System Architecture 32
3.5 Data Model 33
3.6 End-to-End Workflow 33
3.7 Role and Permission Structure 34
3.8 Constraints and Assumptions 34
3.9 Functional Requirement Analysis 34
3.10 Non-Functional Requirements 35
3.11 Principal Use Cases 35
3.12 Detailed Forecast Use Case 36
3.13 Interface Design Principles 36
3.14 Data Ownership and Boundary Rules 36
3.15 Feasibility Analysis 37
3.16 Risk Analysis 37
3.17 Chapter Summary 37
METHODOLOGY AND IMPLEMENTATION 38
4.1 Development and Research Method 38
4.2 Dataset and Feature Dataset 38
4.3 Data Validation and Preprocessing 38
4.4 Feature Engineering 38
4.5 Forecasting Algorithms 39
4.5.1 Linear Regression 39

4.5.2 ARIMA 39
4.5.3 Random Forest 39
4.5.4 Global XGBoost and Seasonal Fallback 39
4.6 Validation and Model Selection 39
4.7 Forecast Request Pipeline 40
4.8 Revenue and Replenishment Calculations 40
4.9 Software Implementation 41
4.10 Security Implementation 41
4.11 Notifications and Reports 42
4.12 Backup, Monitoring and Recovery 42
4.13 Testing Strategy 42
4.14 Chapter Summary 43
4.15 Iterative Development Process 43
4.16 CSV Ingestion Algorithm 43
4.17 Forecasting Procedure 44
4.18 Replenishment Formula Implementation 44
4.19 REST API Design 45
4.20 Web Application Implementation 45
4.21 Mobile Application Implementation 46
4.22 MongoDB Persistence and Indexing 46
4.23 Authentication and Authorization Procedure 46
4.24 Notification Delivery 47
4.25 Deployment and Continuous Verification 47
4.26 Ethical Considerations 47
4.27 Chapter Summary 47
RESULTS AND DISCUSSION 49
5.1 Model Training and Backtest Results 49
5.2 Live Candidate Comparison 49
5.3 Functional Verification 50
5.4 Performance Evaluation 50
5.5 Recovery Verification 51
5.6 Presentation Workspace 51
5.7 Requirements Compliance 51
5.8 Discussion 51

5.9 Limitations 52
5.10 Fold-Level Interpretation 52
5.11 Error Metric Interpretation 52
5.12 Thirty-Day Forecast Persistence Result 53
5.13 Replenishment Case Analysis 53
5.14 Acceptance Test Results 53
5.15 Security Evaluation 54
5.16 Performance Analysis 54
5.17 Reliability and Recovery Analysis 54
5.18 Requirements Traceability Evaluation 55
5.19 Threats to Validity 55
5.20 Chapter Summary 55
SUMMARY, CONTRIBUTIONS AND CONCLUSION 56
6.1 Summary 56
6.2 Main Findings 56
6.3 Contributions 56
6.4 Recommendations 56
6.5 Future Work 57
6.6 Conclusion 57
6.7 Objective-by-Objective Review 57
6.8 Practical Implications 58
6.9 Academic Contribution 58
6.10 Final Conclusion 58
REFERENCES 59
SRS REQUIREMENT COMPLIANCE 60
VERIFICATION EVIDENCE 61
PRINCIPAL REST API ENDPOINTS 62
INTERNAL DEFENCE DEMONSTRATION SEQUENCE 63
Defence Evidence Summary 63
SUBMISSION AND DEPLOYMENT CHECKLIST 64
MONGODB DATA DICTIONARY 65
F.1 Business Collection 65
F.2 User Collection 65
F.3 Membership Collection 65

F.4 Product Collection 65
F.5 Store Collection 65
F.6 Inventory Collection 66
F.7 Sale Collection 66
F.8 Upload Collection 66
F.9 Forecast Collection 66
F.10 Recommendation Collection 66
F.11 Report Collection 67
F.12 Notification Collection 67
F.13 AuditLog Collection 67
F.14 RefreshSession Collection 67
F.15 Invitation Collection 67
F.16 BusinessSettings Collection 68
F.17 PushDevice Collection 68
DETAILED VERIFICATION CATALOGUE 69
G.1 Test Cases 1 to 14 69
G.2 Test Cases 15 to 28 69
G.3 Test Cases 29 to 42 69
G.4 Test Cases 43 to 56 70
G.5 Test Cases 57 to 70 70
G.6 Test Cases 71 to 71 70
API RESOURCE CATALOGUE 71
H.1 Endpoint Group 1 71
H.2 Endpoint Group 2 71
H.3 Endpoint Group 3 71
H.4 Representative Forecast Request 72
H.5 Error Contract 72
USER OPERATIONS GUIDE 73
I.1 Sign In 73
I.2 Review the Dashboard 73
I.3 Maintain Products 73
I.4 Update Inventory 73
I.5 Validate and Upload Sales 73
I.6 Run a Forecast 74

I.7 Review Replenishment 74
I.8 Generate a Report 74
I.9 Manage Notifications 74
I.10 Manage Users 74
I.11 Edit Profile and Security 74
I.12 Mobile Access 75
I.13 Mobile Troubleshooting 75
SECURITY THREAT MODEL 76
J.1 Threat Register 1 76
J.2 Threat Register 2 76
REQUIREMENTS TRACEABILITY MATRIX 77
K.1 Traceability Interpretation 77
OPERATIONAL READINESS CHECKLIST 78
L.1 Defence Demonstration Timing 78
ALGORITHM SPECIFICATIONS 79
M.1 Daily-Series Construction 79
M.2 Holdout Construction 79
M.3 Linear Regression Candidate 80
M.4 ARIMA Candidate 80
M.5 Random Forest Candidate 80
M.6 Global XGBoost Candidate 81
M.7 Seasonal-Naive Candidate 81
M.8 Candidate Selection 81
M.9 Replenishment Algorithm 82
M.10 Report Generation Algorithm 82
DETAILED SYSTEM DESIGN 83
N.1 Architectural Responsibilities 83
N.2 Request Lifecycle 83
N.3 Authentication Sequence 83
N.4 Forecast Sequence 84
N.5 Recommendation Sequence 84
N.6 Reporting Sequence 84
N.7 Notification Sequence 85
N.8 Backup and Restore Sequence 85

N.9 Deployment Topology 85
N.10 Design Trade-offs 85
USER INTERFACE SPECIFICATION 87
O.1 Sign-In Screen 87
O.2 Dashboard 87
O.3 Products 87
O.4 CSV Upload 87
O.5 Forecasting 88
O.6 Recommendations and Insights 88
O.7 Reports 88
O.8 Administration 88
O.9 Profile and Settings 89
O.10 Mobile Adaptation 89
O.11 Accessibility and Visual Consistency 89
PROJECT PLANNING AND MANAGEMENT 90
P.1 Work Breakdown 90
P.2 Version and Configuration Control 90
P.3 Change Control 90
P.4 Risk Monitoring 91
QUALITY ASSURANCE AND MAINTENANCE 92
Q.1 Quality Strategy 92
Q.2 Defect Classification 92
Q.3 Regression Verification 92
Q.4 Monitoring and Incident Response 92
Q.5 Maintenance Schedule 93
Q.6 Model Maintenance 93
Q.7 Database Maintenance 93
Q.8 Client Maintenance 93
GLOSSARY 95
R.1 Terms 1 to 20 95
R.2 Terms 21 to 40 95
R.3 Terms 41 to 59 95
DETAILED SRS REQUIREMENT SPECIFICATIONS 97
S.1 REQ-1 User Registration 97

S.2 REQ-2 Secure Login 97
S.3 REQ-3 Protected Business Data 97
S.4 REQ-4 CSV Validation and Import 97
S.5 REQ-5 Product Demand Forecast 98
S.6 REQ-6 Required Forecasting Models 98
S.7 REQ-7 Metric Comparison and Selection 98
S.8 REQ-8 Authorized Forecast Access 98
S.9 REQ-9 Product and Inventory Management 98
S.10 REQ-10 Revenue and Replenishment Insight 99
S.11 REQ-11 Responsive Analytics 99
S.12 REQ-12 Operational Alerts 99
S.13 REQ-13 Administration 99
S.14 REQ-14 Audit Retention 100
S.15 REQ-15 Role-Based Access 100
S.16 REQ-16 Graceful Error Handling 100
S.17 REQ-17 Backup and Recovery 100
S.18 REQ-18 Privacy and Secure Communication 100
WORKED BUSINESS SCENARIO 102
T.1 Scenario Context 102
T.2 Product and Inventory Setup 102
T.3 Historical Sales Import 102
T.4 Thirty-Day Forecast 102
T.5 Replenishment Decision 103
T.6 Reporting and Insight 103
T.7 Mobile Review 103
T.8 Administrative Review 103
T.9 Operational Evidence 104
EVIDENCE REGISTER 105
U.1 Model Evidence 105
U.2 Application Performance Evidence 105
U.3 Recovery Evidence 105
U.4 Artifact Integrity Evidence 105
U.5 Evidence Limitations 106
INTERNAL DEFENCE QUESTIONS AND EVIDENCE 107

V.1 Why were several forecasting models implemented? 107
V.2 Why is MAE the primary selection metric? 107
V.3 How is data leakage prevented? 107
V.4 What makes the thirty-day requirement complete? 107
V.5 How is a forecast converted into an order quantity? 107
V.6 How is business data isolated? 107
V.7 What happens if the Python service is unavailable? 108
V.8 What evidence shows production readiness? 108

List of Figures

Figure 1: StockGuard AI system architecture

Figure 2: Product-level forecasting workflow

Figure 3: Replenishment decision flow

Figure 4: Three-fold recursive XGBoost backtest

Figure 5: Verified production performance

Figure 6: Deployment, monitoring and recovery workflow

List of Tables

Table 1: Project objectives and completion evidence

Table 2: Forecasting model comparison

Table 3: Research gap and StockGuard response

Table 4: Delivered functional areas

Table 5: Core MongoDB entities

Table 6: Role and permission structure

Table 7: Feature engineering variables

Table 8: Technology stack

Table 9: Three-fold recursive backtest results

Table 10: Functional verification summary

Table 11: SRS requirement compliance

Table 12: Principal REST API endpoints

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized enterprises contribute employment, innovation and local economic activity, yet many operate without specialised forecasting or inventory-planning teams. Sales history is commonly stored in spreadsheets or point-of-sale exports, while replenishment decisions depend on recent experience and informal judgement. Such methods may be adequate for a stable product, but they become unreliable when a business manages multiple stores, seasonal items and products with different selling rates.

Inventory decisions involve a direct trade-off. Insufficient stock produces lost sales, delayed orders and dissatisfied customers. Excess stock ties up working capital, occupies storage space and may become obsolete. The decision is more difficult because demand changes over time and because the business must order before future demand becomes observable. A useful system therefore needs to estimate demand, communicate uncertainty and translate the estimate into a concrete action.

Modern data analytics makes it possible to learn patterns from historical transactions. Statistical models can represent trend and seasonality, while machine-learning models can capture nonlinear relationships among lagged demand, rolling averages and calendar variables. A business intelligence layer can then convert predictions into revenue, stock-risk and replenishment information. StockGuard AI brings these functions together in a SaaS-style application intended for non-technical SME users.

1.2 Project Motivation

The proposal identified that enterprise forecasting products are often costly, difficult to configure or too broad for small organisations. An SME needs a focused workflow: create a product catalogue, upload historical sales, request a product-level forecast, compare models and decide how much to order. The solution should work in a browser and on a mobile device, support multiple users and protect each business workspace from other tenants.

A second motivation was explainability at the operational level. Forecast accuracy alone does not tell a purchasing officer what to do. StockGuard AI therefore combines the selected forecast with supplier lead time, current and reserved inventory, historical variability and a configurable safety-stock policy. The system explains why it recommends an order, highlights urgency and records the action lifecycle.

1.3 Problem Statement

SMEs possess sales data but frequently lack an affordable, secure and understandable system for transforming it into product-level demand forecasts and replenishment decisions. Manual estimates and disconnected spreadsheets can produce misleading forecasts, inconsistent stock policies, weak revenue planning and delayed responses to low stock or demand spikes. Existing tools may omit product-level model comparison, cross-platform access, multi-user control or direct stock recommendations.

The project problem is therefore to design and implement a cross-platform predictive business intelligence system that accepts validated sales histories, compares appropriate forecasting models,

stores complete future predictions, converts those predictions into business actions and provides the result to authorized users within the performance and security limits stated in the SRS.

1.4 Aim and Objectives

The main aim is to provide SMEs with a practical forecasting and inventory decision-support platform that is accurate enough for product-level planning, simple enough for routine use and secure enough for multi-user business data.

Table 1: Project objectives and completion evidence

Objective	Delivered evidence
Generate demand forecasts	Product and store series support 7, 14, 28 and 30-day horizons.
Compare forecasting candidates	Linear Regression, ARIMA, Random Forest, XGBoost and seasonal fallback paths expose MAE/RMSE evidence.
Support inventory decisions	Lead-time demand, variability, safety stock, target stock and order quantity are calculated.
Deliver web and mobile access	React/Vite and React Native/Expo clients share the protected business API.
Protect multi-user data	JWT sessions, role permissions, tenant filters and audit logs are implemented.
Verify operational quality	Automated checks, load tests, health monitoring and encrypted recovery evidence are included.

1.5 Research Questions

1. How can multiple forecasting methods be compared fairly on the same product-level history?
2. How can a forecast be converted into a repeatable stock-replenishment recommendation?
3. How can SME users access analytical functions through web and mobile interfaces without losing security or tenant isolation?
4. Can the deployed prototype satisfy the response-time, concurrency, forecasting and recovery targets in the submitted SRS?

1.6 Significance of the Study

The project demonstrates applied integration rather than a single isolated model. It connects data validation, time-series modelling, inventory logic, access control, visualisation and deployment in one working system. This matters because an accurate notebook experiment is not automatically an operational product. The implemented API, persistence rules, user workflows and recovery controls show how predictive analytics can become a business process.

For SMEs, the practical contribution is a clear path from historical sales to an order decision. For software engineering, the project demonstrates modular separation between user interfaces, business services, forecasting services and storage. For academic evaluation, the report provides traceable evidence between the proposal, SRS requirements, implementation and measured results.

1.7 Scope and Delimitations

The delivered scope includes registration and authentication; role-based access; profile and settings management; product, store, inventory and sales records; CSV upload and validation; demand and revenue forecasting; model comparison; replenishment recommendations; dashboards, reports and insights; notifications; administration; audit logging; web and mobile clients; production deployment; monitoring; encrypted backup and restore verification.

The prototype is limited to historical business sales and configured inventory inputs. It does not automatically ingest point-of-sale streams, supplier portals, promotions, weather or macroeconomic data. SMTP credentials and platform store credentials are external operational settings. Forecast quality remains dependent on the continuity, relevance and accuracy of each business dataset.

1.8 Structure of the Report

Chapter 2 reviews forecasting, evaluation, inventory and business intelligence concepts. Chapter 3 defines the problem and proposed framework. Chapter 4 explains the methodology and implementation. Chapter 5 presents model, functional, performance and recovery results. Chapter 6 summarises contributions, recommendations, limitations, future work and the conclusion. Appendices provide requirements, tests, API routes and a defence demonstration sequence.

1.9 SME Inventory Management Context

Inventory planning in an SME differs from planning in a large retail enterprise. A small organisation may have fewer products, but it often has less purchasing power, a smaller cash buffer and fewer specialist employees. One incorrect order can therefore affect a greater proportion of working capital. A stockout can also be more damaging because customers may have only a limited reason to return when a common product is unavailable. These conditions make an understandable and inexpensive forecasting workflow especially valuable for smaller firms.

Daily decisions are usually distributed among an owner, a store manager and operational staff. The owner is interested in revenue, cash committed to stock and the overall health of the business. A manager needs product-level demand, days of cover and a defensible order quantity. Staff members need a clear way to record products, inventory and sales. StockGuard AI separates these responsibilities through roles and permissions while retaining one shared business workspace. This arrangement reflects the way an SME actually works rather than presenting every user with the same unrestricted interface.

Spreadsheet-based forecasting creates several practical risks. Formulas may be overwritten, date formats may change between files and copied worksheets can silently omit new rows. Forecast logic is difficult to audit when it is spread across multiple tabs. A web service can validate the input once, preserve the original upload metadata and store each forecast as a versioned record. The user can then return to the same result from the web or mobile client without reconstructing the calculation.

StockGuard AI treats forecasting as one stage in a broader decision process. The forecast estimates what is likely to sell; it does not by itself state what should be purchased. Replenishment also depends on current inventory, stock reserved for existing commitments, supplier lead time, demand variability and the organisation's preferred safety margin. The system therefore keeps prediction and recommendation as linked but distinct records. This distinction improves explainability because a user can inspect both the forecast and the operational assumptions that produced an order recommendation.

1.10 Stakeholders and Expected Benefits

Table 2: Stakeholder needs and expected project benefits

Stakeholder	Primary need	Expected benefit
Business owner	Understand revenue, stock exposure and future demand	Faster decisions and improved use of working capital
Administrator	Control users, roles, permissions and activity	Accountability and reduced unauthorized access

Stakeholder	Primary need	Expected benefit
Store manager	Know which products require action	Prioritized replenishment and fewer avoidable stockouts
Analyst	Compare models and examine performance	Evidence-based selection using MAE and RMSE
Operational staff	Maintain products, sales and inventory	Consistent records and immediate validation feedback
FYP evaluators	Trace claims to implementation evidence	A reproducible demonstration and requirements matrix

The immediate benefit is decision speed. Instead of manually grouping sales, calculating an average and estimating the remaining stock, an authorized user can request a forecast and receive a stored sequence of future daily values. The related recommendation then expresses the result as a quantity to review. This does not remove managerial judgement; it concentrates attention on the products where judgement is needed most.

A second benefit is consistency. The same validation, model comparison and replenishment formulas apply regardless of whether the request originates from the web client or the mobile application. Common server-side rules avoid a situation in which two interfaces produce different answers from the same data. Consistent tenant filters also ensure that analytics, reports and notifications remain within the authenticated business workspace.

A third benefit is traceability. A completed forecast records the input series, horizon, candidate metrics, selected model, predicted values and financial totals. Administrative and security actions create audit events. Recommendations retain their status history. These records allow an evaluator or business owner to ask not only what the current result is, but also when it was generated, which model was chosen and what action followed.

1.11 Operational Scenario

Consider an SME that manages several product categories and receives a weekly supplier delivery. At the beginning of a planning cycle, a manager uploads recent sales exported from the point-of-sale system. StockGuard AI validates the column names, dates, quantities and duplicate identifiers before any row is committed. Invalid rows are reported with their location and reason so that the source file can be corrected rather than partially imported without the user's knowledge.

After the data is accepted, the manager opens a product and requests a 30-day forecast. The API constructs a continuous daily history, fills missing calendar dates with zero sales where appropriate and sends the common history to available forecasting candidates. Linear Regression, ARIMA and Random Forest are evaluated by the Python service. The Node pipeline can additionally use the verified XGBoost model when the store and product identifiers have trained mappings, and it retains a seasonal-naive candidate for resilience.

The candidate with the lowest holdout MAE is selected, with RMSE used as the tie-breaker. Thirty dated predictions are stored rather than only a total. The user can therefore inspect the shape of expected demand and not merely a single aggregate. The system multiplies the predicted quantities by the current product price to estimate revenue and then calculates lead-time demand, safety stock, target stock and recommended order quantity.

If the available inventory cannot cover expected demand, a low-stock or critical recommendation is opened and a notification is created. The manager reviews the explanation, approves or dismisses the recommendation and may later mark it completed when stock has been ordered or received. Reports

and insights include the same saved result. This scenario represents the complete chain from uploaded evidence to an auditable business action.

1.12 Success Criteria

1. An authenticated user can maintain products, stores, inventory and sales within an isolated business workspace.
2. A CSV file is rejected before import when it exceeds limits or contains invalid headers, dates, quantities or duplicate business keys.
3. A user can request 7, 14, 28 or 30 daily predictions and retrieve the complete saved sequence later.
4. Linear Regression, ARIMA and Random Forest are compared on the same holdout period using MAE and RMSE.
5. Eligible series can use the verified global XGBoost bundle, while an explicit seasonal fallback prevents a single candidate failure from ending the request.
6. Forecast totals are converted into explainable replenishment quantities and risk classifications.
7. Web and mobile clients display the same protected MongoDB records through the common API.
8. Owners and administrators can manage users and review auditable activity without receiving access to another tenant.
9. The deployed service meets the response-time, concurrent-user and recovery objectives recorded in the SRS.

1.13 Definitions and Abbreviations

Table 3: Principal terms used in this report

Term	Meaning in StockGuard AI
SME	Small or medium-sized enterprise using the platform as an isolated tenant
SKU	Stock keeping unit that uniquely identifies a product within the business
Forecast horizon	Number of future daily values requested: 7, 14, 28 or 30
MAE	Mean absolute error, used as the primary model-selection metric
RMSE	Root mean squared error, reported and used to resolve an MAE tie
Lead time	Expected number of days between ordering and receiving stock
Safety stock	Additional inventory intended to absorb demand variability
Target stock	Inventory level required for lead-time and review-period coverage
Tenant	Business workspace whose records are isolated from other businesses
RBAC	Role-based access control enforced by API middleware
JWT	Signed access token representing an authenticated session
RPO / RTO	Recovery point and recovery time objectives used for continuity planning

1.14 Chapter Summary

This chapter established the business need, research motivation, project objectives, stakeholders and success criteria for StockGuard AI. It also explained why a useful solution must join forecasting with validated data, inventory context, secure access and an auditable recommendation workflow. The following chapter reviews the forecasting, inventory and software-engineering literature that informed the final design.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

Demand forecasting estimates future sales from historical observations and relevant explanatory variables. In inventory management, the forecast supports decisions made before demand occurs. The literature includes statistical time-series methods, supervised machine-learning methods and ensemble models. Their usefulness depends on the data pattern, history length, forecast horizon, computational cost and ability to generalise.

2.2 Linear Regression

Linear Regression represents the target as a weighted combination of input variables. For product demand, the variables may include lags, rolling averages and cyclical calendar features. The method is fast, reproducible and easy to inspect, which makes it a valuable baseline. Its central limitation is that a fixed linear relationship cannot naturally represent every nonlinear demand pattern or interaction.

StockGuard uses Linear Regression as one candidate in the Python service. The candidate receives the same prepared history and validation split as the other methods. This prevents a favourable data split from giving one algorithm an artificial advantage and allows its MAE and RMSE to be interpreted beside other models.

2.3 ARIMA

Autoregressive Integrated Moving Average models combine autoregression, differencing and moving-average error terms. They are established tools for univariate time-series forecasting and can represent temporal dependence without requiring a large feature matrix. Their assumptions and parameter selection require care, particularly when a series is short, intermittent or changes structure.

ARIMA is included because some sales series are primarily driven by their own recent values and temporal structure. In StockGuard, it is evaluated at request time for the selected product/store series. If the model cannot be estimated reliably for a short or unsuitable series, the service returns a controlled candidate error and the comparison continues with the remaining models.

2.4 Random Forest and Gradient-Boosted Trees

Random Forest combines multiple decision trees trained on resampled data and random feature subsets. Averaging reduces the variance of a single tree and makes the method effective for nonlinear relationships. It can use lag, rolling and calendar variables without requiring a predetermined linear form. Its predictions remain dependent on patterns represented in the training history and may be conservative beyond the observed range.

Gradient-boosted trees build an additive sequence of learners, with each stage focusing on the remaining error. XGBoost provides regularisation, efficient tree construction and portable model serialization. StockGuard extends the three proposal models with a verified global XGBoost bundle trained on millions of store/product/day observations. This production candidate is used where category mappings and sufficient features are available; otherwise the candidate pipeline can use the seasonal fallback and Python models.

2.5 Model Evaluation

Forecast evaluation must preserve temporal order. Randomly shuffling observations can expose future information to the training set and overstate accuracy. StockGuard uses holdout periods and recursive forecasting: a model is trained on observations available before a cutoff, predicts the future period, and is compared with the actual values. Repeating this process at several cutoffs measures stability across time.

Mean Absolute Error (MAE) is the average absolute difference between actual and predicted demand. Root Mean Squared Error (RMSE) takes the square root of the mean squared error and penalises large mistakes more heavily. The application selects the candidate with the lowest MAE and uses RMSE as a tie-breaker. Both measures are retained so users can distinguish consistent small errors from occasional large deviations.

Table 4: Forecasting model comparison

Model	Strength	Limitation	Role in StockGuard
Linear Regression	Fast and interpretable baseline	Limited nonlinear representation	Per-series candidate
ARIMA	Explicit temporal dependence	Sensitive to short or unstable series	Per-series time-series candidate
Random Forest	Nonlinear relationships and interactions	Can be conservative outside observed range	Per-series ML candidate
Global XGBoost	Scalable tree ensemble with learned cross-series patterns	Requires matching feature/category mappings	Verified production candidate
Seasonal naive	Robust and transparent fallback	Repeats past seasonal behaviour	Fallback and benchmark

2.6 Forecast-Based Inventory Control

A demand forecast becomes operational when it is linked with lead time and stock position. Lead-time demand estimates the units expected before a replenishment order arrives. Safety stock protects against variability and forecast error. The reorder point identifies the stock level at which an order is needed, while target stock represents the desired inventory after replenishment.

Textbook policies often use a service-level factor multiplied by demand standard deviation and the square root of lead time. Practical businesses may also define a minimum safety-stock percentage. StockGuard calculates both a variability-based amount and a policy amount, then uses the larger. This makes the recommendation responsive to observed volatility while respecting the business setting.

2.7 Business Intelligence and Visualisation

Business intelligence combines data preparation, analysis and presentation to support decisions. Forecasts are easier to act on when users can see recent history, future daily values, revenue implications, category behaviour and stock risk in the same workspace. Interactive dashboards reduce the effort needed to inspect a single product, while reports create a shareable record of the analysis.

A useful interface must distinguish an estimate from an action. StockGuard therefore separates forecast outputs from recommendation status. Users can review the calculation, approve an action, dismiss it when business context contradicts the model, or mark it completed after purchasing. Audit events make these decisions reviewable.

2.8 Security and Multi-Tenant SaaS

A multi-tenant SaaS application stores records for several organisations in shared infrastructure. Correct isolation depends on applying the authenticated workspace identifier to every business query,

not merely hiding data in the interface. Authentication, authorization and tenant filtering are separate controls: a valid user must still possess permission for the route, and the query must still be restricted to the user's business.

StockGuard applies bcrypt password hashing, short-lived JWT access tokens, revocable refresh sessions, role/permission middleware and tenant-scoped Mongoose queries. Web refresh credentials use an HttpOnly cookie, whereas the mobile client stores its refresh token in Expo SecureStore. Password changes and resets rotate a session version so previously issued sessions become invalid.

2.9 Research Gap

The reviewed approaches often solve one layer of the problem: a notebook compares models, an inventory package records quantities, or a dashboard visualises past sales. The project gap is an affordable cross-platform workflow that connects validated SME data to model comparison, complete 30-day predictions, revenue estimates, replenishment actions, role-controlled collaboration, notifications and evidence-based recovery.

Table 5: Research gap and StockGuard response

Observed gap	StockGuard AI response
Manual estimates and disconnected spreadsheets	Validated upload, persistent product/store series and repeatable forecast runs
One model used for every series	LR, ARIMA, RF, XGBoost and seasonal candidates with MAE/RMSE selection
Forecast without a purchasing action	Safety stock, target stock, order quantity and recommendation lifecycle
Desktop-only or analyst-only tools	Responsive web and React Native mobile application
Weak separation between businesses	JWT, RBAC, permission checks and tenant-scoped queries
No operational recovery evidence	Encrypted scheduled backups and isolated restore verification

2.10 Chapter Summary

This chapter reviewed the five forecasting paths used by StockGuard AI, the temporal validation approach, the role of MAE and RMSE, and the inventory concepts required to convert predicted demand into a purchasing decision. Linear Regression offers an interpretable baseline, ARIMA captures univariate time dependence, Random Forest represents nonlinear relationships, XGBoost provides a verified global ensemble, and seasonal naive supplies a robust fallback.

The review also established that forecasting accuracy, application usability and data protection must be considered together. The identified research gap is an SME-oriented system that connects model comparison with complete forecast persistence, replenishment recommendations, role-controlled collaboration and operational recovery. Chapter 3 translates that gap into the proposed StockGuard framework.

2.10 Characteristics of Retail Demand

Retail demand is rarely represented by a single stable statistical pattern. A mature everyday product may exhibit a consistent level with weekly seasonality, while a fashion item may rise rapidly and then decline. Some products sell every day; others are intermittent and contain many zero-demand dates. A forecasting system intended for mixed SME catalogues must therefore avoid assuming that one algorithm is universally superior.

Level describes the typical magnitude of a series, trend describes persistent movement and seasonality describes a pattern tied to a recurring calendar interval. Cycles differ from seasonality

because their duration is not fixed. Noise represents variation that cannot be explained by the selected structure. These components influence model behaviour: regression can express a deterministic trend, ARIMA can model autocorrelation after differencing, tree ensembles can learn nonlinear interactions and seasonal-naive forecasting can preserve a strong repeating weekly pattern.

Intermittent demand presents a special difficulty because common percentage errors are undefined or unstable when actual demand is zero. StockGuard AI consequently uses MAE and RMSE rather than mean absolute percentage error as its core comparison measures. MAE remains interpretable in units sold, while RMSE places greater weight on unusually large errors. Reporting both values helps a user distinguish consistent moderate error from occasional large misses.

Aggregation level also affects apparent predictability. Total category demand is usually smoother than a single product-store series because positive and negative variations can cancel. Replenishment, however, requires product-level information. The system therefore forecasts at the operational series level and aggregates stored results for dashboards and reports. This bottom-up approach preserves actionability while still supporting business-level summaries.

2.11 Time-Series Preparation

A model cannot interpret an absent date unless the software first creates a continuous calendar. StockGuard AI groups uploaded transactions by date, product and store, sums quantities for the same business key and reindexes the series across the complete date range. Missing dates inside that observed range become zero sales when the uploaded dataset indicates no transaction, whereas malformed or missing date fields are rejected during validation. This separation prevents data-quality errors from being silently treated as genuine zero demand.

Chronological order must be preserved throughout preparation. A random train-test split would allow future behaviour to influence a model evaluated on the past and would overstate real forecasting accuracy. The implementation holds out the most recent part of the series, trains each candidate only on earlier observations and compares predictions with the held-out actual values. Recursive backtesting extends this idea across multiple cut-off dates and is used for the production XGBoost evidence.

Outliers may represent either an error or a real event. Automatically removing every spike can erase promotions, bulk orders or shortages that matter to the business. StockGuard AI emphasizes validation of impossible values, such as negative quantities where returns are not supported, while preserving legitimate high sales. Rolling features and tree ensembles reduce the influence of isolated observations without rewriting the source history.

2.12 Feature Engineering for Demand Forecasting

Lag variables convert temporal dependence into supervised-learning inputs. A one-day lag represents immediate persistence; seven and fourteen-day lags capture weekly repetition; and a twenty-eight-day lag supplies a four-week reference. Because a prediction for tomorrow becomes part of the information needed for later future dates, multi-day production forecasting is recursive. The system appends each generated value to the working history before constructing the next row of lag features.

Rolling averages describe local level while reducing day-to-day noise. A seven-day mean represents the most recent week and a twenty-eight-day mean represents a broader monthly baseline. The features are shifted so the target date never contributes to its own predictors. This is an essential

leakage control: an apparently accurate model that uses the target value inside a rolling average would fail when asked to forecast a genuinely unknown date.

Calendar variables encode systematic timing effects. Direct weekday integers incorrectly imply that Sunday is far from Monday, even though the days are adjacent on a weekly cycle. Sine and cosine transformations place the beginning and end of a cycle next to each other geometrically. StockGuard AI applies paired cyclical features for day and month, allowing linear and tree-based candidates to learn recurring timing patterns without an artificial boundary.

The global XGBoost model also uses encoded store and product identifiers. Category mappings are stored with the model bundle so inference uses the same codes as training. An identifier absent from those mappings is not forced into an arbitrary trained category; the pipeline uses the local candidates and seasonal fallback instead. This explicit eligibility rule protects the semantic meaning of the model input.

2.13 Forecast Evaluation and Selection

Forecast evaluation must reflect the decision the model will support. A small average error may still hide an occasional large underestimate that creates a stockout. MAE provides a stable average deviation, while RMSE increases the influence of large residuals. StockGuard AI orders candidates by MAE and uses RMSE when two MAE values are effectively equal. The chosen result and all candidate metrics are stored so that selection remains inspectable.

A holdout score is an estimate rather than a permanent property of a model. Performance can change when the product enters a different season, when price changes or when customer behaviour shifts. The platform therefore stores the metric with each run rather than displaying one static accuracy percentage for every product. Re-running a forecast after new data arrives creates a new evidence point and supports later drift analysis.

Model comparison should use identical data and horizons. If one candidate is evaluated on an easier time period, its score cannot be compared fairly with another. The Python service creates a common holdout window and returns candidate-level failures individually. A model that cannot fit the available history is marked unavailable; the remaining valid candidates can still produce a result. This design is more robust than treating an ARIMA convergence warning, for example, as failure of the entire forecasting feature.

2.14 Inventory Control Concepts

Forecasting answers a demand question, whereas inventory control answers a timing and quantity question. Lead-time demand is the expected quantity sold before a replenishment can arrive. If current usable inventory is below that amount, the firm is exposed even when total monthly inventory appears adequate. The system therefore sums the predictions that fall inside the supplier lead-time window rather than dividing the full forecast total mechanically.

Safety stock protects against variation around the expected value. Classical approaches often multiply a service factor by the standard deviation of demand during lead time. StockGuard AI uses observed daily variability, the square root of lead time and the configured safety percentage to create an explainable buffer. The design favours transparency for an SME prototype and exposes the underlying quantities so a manager can assess whether the setting is reasonable.

Target stock extends coverage beyond the arrival date. It combines lead-time demand, safety stock and demand expected during the review period. Recommended order quantity is the non-negative gap between target stock and available stock after reserved units are considered. A zero recommendation is meaningful: it indicates that the current position already covers the selected policy, even if a future forecast exists.

Overstock and demand-spike signals complement low-stock logic. Overstock indicates that days of cover or available units materially exceed expected demand. A demand spike compares near-future predictions with the recent baseline and can prompt earlier review even when the product is not yet critical. Separating recommendation types lets the interface prioritize risk and lets the user dismiss one conclusion without deleting the underlying forecast.

2.15 Business Intelligence for SMEs

Business intelligence systems combine operational records into information that supports planning. For a small organisation, usefulness depends on reducing cognitive load. A dashboard should reveal total revenue, sales activity, stock exposure and urgent recommendations without requiring the user to understand database queries. Detailed product and forecast pages can then support investigation. StockGuard AI follows this progressive-disclosure pattern across desktop and mobile interfaces.

Visual design can mislead when axes, units or time windows are unclear. The implementation labels quantity and currency measures, uses stored dates on forecast charts and preserves model names with their metrics. Reports use saved records rather than recalculating a different figure in the client. These decisions improve consistency between dashboard cards, downloadable reports and the data returned by the API.

Insights in StockGuard AI are rule-supported interpretations of calculated evidence. The system may identify stockout risk, a rising forecast or an overstock position, but it does not claim a causal explanation that the data cannot support. This distinction is important in an academic system: prediction estimates future observations, while business interpretation explains the operational implication of that estimate under explicit assumptions.

2.16 Software Architecture and Cross-Platform Delivery

A layered web architecture separates presentation, business logic, persistence and specialist model execution. React and React Native render interfaces suited to their platforms. Express centralizes authentication, validation, permissions and domain operations. MongoDB stores tenant-scoped documents, while the Python service hosts libraries commonly used for statistical forecasting. The Node process remains the public API boundary and can continue with eligible candidates if the Python service is temporarily unavailable.

Sharing an API between web and mobile clients reduces duplicated logic. Both clients submit the same forecast request and receive the same saved record. Mobile-specific responsibilities are limited to navigation, secure token storage, device push registration and compact presentation. This arrangement also simplifies testing because server integration tests validate business behaviour independently of a particular screen.

Cloud deployment introduces cold starts, network latency and secret management. The production design uses environment variables for credentials, health checks for dependencies and automated workflows for monitoring and backup. Static model artifacts are verified by hash before use. These

controls make the deployed prototype more defensible than a local-only demonstration even though a full commercial service would require longer-term operational evidence.

2.17 Security and Privacy Literature Applied to the Project

Authentication confirms identity, authorization determines permitted actions and tenant isolation limits the records to which an authorized action applies. These are separate controls. A valid analyst token, for example, should not allow user suspension, and a permitted product read must still include the business identifier. StockGuard AI combines signed access tokens, permission middleware and tenant-scoped database queries to enforce all three dimensions.

Refresh credentials require stronger handling because they can create new access tokens. The web client uses an HttpOnly cookie so browser scripts cannot read the refresh value, while the mobile application uses secure device storage. Session versions and revocable refresh records allow password changes and resets to invalidate older sessions. This is more robust than accepting a long-lived bearer token until it expires.

Audit data supports accountability but can itself contain sensitive metadata. The project stores action, actor, target, time and limited request context rather than raw passwords or tokens. A one-year expiry policy exceeds the submitted ninety-day minimum while preventing indefinite retention. Backup artifacts use authenticated encryption so confidentiality and tamper detection are checked together during recovery verification.

2.18 Comparative Synthesis

Table 6: Comparative synthesis of techniques considered

Technique	Strength	Limitation	Use in StockGuard AI
Linear Regression	Transparent and fast trend baseline	Limited nonlinear capacity	Live candidate for suitable series
ARIMA	Models autocorrelation and differenced trend	Sensitive to order and short histories	Live statistical candidate
Random Forest	Nonlinear and robust to interactions	Recursive forecasts can accumulate error	Live machine-learning candidate
XGBoost	Efficient global tree boosting	Requires trained identifier mappings	Verified production candidate when eligible
Seasonal naive	Simple, stable and interpretable	Cannot adapt quickly to structural change	Explicit fallback and benchmark
Safety stock	Adds variability protection	Depends on service policy and data quality	Configurable replenishment buffer
MERN architecture	One API and flexible document model	Requires disciplined tenant filtering	Core business application stack
Python service	Mature statistics and ML ecosystem	Additional deployed dependency	Hosts LR, ARIMA and RF comparison

2.19 Refined Research Gap

Existing forecasting tutorials frequently stop after reporting a model score, while inventory applications may use a fixed reorder threshold without a validated future-demand estimate. Enterprise planning suites can combine these functions but may be too costly or complex for the target SME. The practical gap is therefore an affordable workflow that validates ordinary CSV data, compares understandable models, preserves every daily prediction and turns the result into a reviewable order quantity.

A second gap concerns implementation evidence. A prototype can display plausible charts while relying on hard-coded data or unprotected endpoints. StockGuard AI addresses this gap through saved MongoDB records, authenticated production checks, tenant-isolation tests, concurrent-load evidence and isolated restore verification. The research contribution is not a new forecasting algorithm; it is the integration and verification of forecasting, inventory logic and cross-platform business use.

2.20 Chapter Summary

The literature supports a multi-model approach because product demand exhibits different statistical patterns. It also shows that accuracy metrics must be tied to chronological evaluation and that a forecast becomes operationally useful only when combined with inventory policy. The reviewed software and security concepts justify a layered, tenant-isolated architecture. These findings lead directly to the proposed framework described in Chapter 3.

CHAPTER 3
PROBLEM STATEMENT AND PROPOSED FRAMEWORK

3.1 Formal Problem Definition

Given a sequence of dated sales observations for a product at a store, current and reserved inventory, unit price, supplier lead time and workspace policy settings, StockGuard must estimate future daily demand for a requested horizon. It must compare eligible candidates on a historical validation window, select the candidate with the lowest measured error, store the selected predictions and convert the forecast into revenue and replenishment information.

The system must provide this function only to authorized members of the owning business, support up to 10,000 uploaded rows and a 10 MB CSV, display meaningful validation errors, maintain audit history, and operate on contemporary web, Android and iOS clients. The application must remain usable when an optional integration such as SMTP is not configured.

3.2 Stakeholders and User Classes

The principal stakeholder is the SME business owner who requires a consolidated view of products, sales, forecast risk and purchasing actions. Administrators manage users, invitations, roles, permissions and system activity. Managers supervise products, inventory and recommendations. Analysts explore forecasts, reports and insights. Staff members perform permitted operational tasks. The project team and system operators maintain deployment, monitoring, backups and model services.

3.3 Delivered Functional Areas

Table 7: Delivered functional areas

Area	Implemented capability
Identity	Registration, login, refresh, logout, password change/reset, profile
Products and inventory	Product CRUD, stores, stock quantities, lead time and pricing
Data ingestion	CSV header/type/date/duplicate checks, idempotent import and rollback
Forecasting	7/14/28/30-day candidate comparison and saved daily predictions
Decision support	Revenue, reorder point, safety stock, target stock and order quantity
Business intelligence	Dashboard, reports, insights and category/product analytics
Administration	Invitations, users, suspension, roles, permissions and audit logs
Notifications	Upload, forecast, low-stock, critical-stock and recommendation events
Operations	Health checks, encrypted backup, restore verification and deployment profiles

3.4 Proposed System Architecture

The framework uses a layered architecture so that user interfaces, business rules, model execution and persistence can evolve independently. The React web client and React Native mobile client communicate with the same Express API. The API authenticates the request, validates data, enforces permissions and business isolation, orchestrates forecasting services and persists results in MongoDB Atlas.

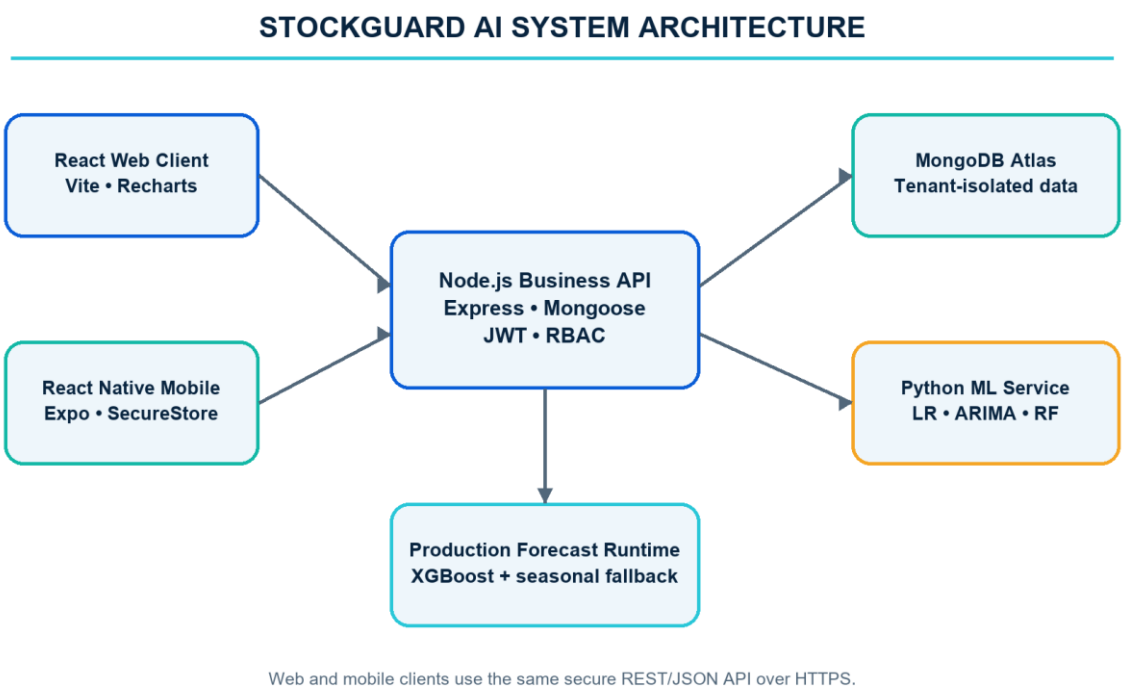


Figure 1: StockGuard AI system architecture

The Python service provides Linear Regression, ARIMA and Random Forest candidates. The Node runtime loads the verified XGBoost JSON model and its category mappings and provides a seasonal fallback. The API normalizes candidate results, selects the best valid model and applies decision-support logic. Vercel hosts the production web/API functions, while GitHub Actions performs scheduled health and recovery tasks. Expo Application Services provides mobile build infrastructure.

3.5 Data Model

MongoDB collections are organised around a Business tenant. Each operational document stores a business identifier, and service methods include that identifier in queries and writes. This pattern supports shared infrastructure without mixing business records.

Table 8: Core MongoDB entities

Entity	Purpose	Important relationships
Business	Workspace settings and defaults	Owns users, stores, products and all analytical records
User	Identity, role, permissions and session version	Belongs to one business
Store	Business location/code	Owns inventory and sales series
Product	SKU, category, price and supplier lead time	Referenced by sales, inventory and forecasts
Sale	Dated quantity and revenue	Links business, store and product
ForecastRun / Forecast	Model metrics, totals and daily predictions	Links series and selected candidate
Inventory	On-hand and reserved stock	Unique by product/store/workspace
Recommendation	Risk, quantity, reason and lifecycle	Derived from a forecast and inventory plan
Notification	Persistent user alert	May refer to upload, forecast or recommendation
AuditLog	Security and activity event	Records actor, action, metadata and timestamp

3.6 End-to-End Workflow

An authorized user creates or selects a product and store, then uploads historical sales in the required CSV format. The API validates the file before committing records. A forecast request identifies the product, store and horizon. The API loads the series and inventory context, invokes eligible candidates and stores the chosen result. Decision support calculates replenishment measures

and creates or updates recommendations. The clients then display the forecast, recommendations, reports and alerts.

3.7 Role and Permission Structure

Table 9: Role and permission structure

Role	Typical access
Owner	Full workspace, user, security, configuration and analytical access
Administrator	User management, audit logs, products, uploads, forecasts and reports
Manager	Products, inventory, forecasts, recommendations and reports
Analyst	Uploads, model comparison, forecasts, reports and insights
Staff	Assigned operational read/write actions only

Roles provide sensible defaults, while custom permission arrays allow an owner or administrator to adjust access for a specific user. Backend middleware remains the authority; hiding a navigation item in the client is only a usability measure and does not grant or revoke API access.

3.8 Constraints and Assumptions

- Forecasts assume that the uploaded product/store history is correctly dated and represents the business process being forecast.
- Products with insufficient history may require the seasonal fallback or may produce wider practical uncertainty.
- The XGBoost candidate requires the imported category mappings and the twelve training features used by the model bundle.
- Email delivery depends on external SMTP credentials; in-app notifications remain available without SMTP.
- Physical iOS and Android distribution depends on Apple/Google developer credentials outside the source repository.
- Cloud availability depends on Vercel, MongoDB Atlas and the configured Python service, with health checks used to detect failures.

3.9 Functional Requirement Analysis

Functional requirements were derived from the submitted SRS and then traced to routes, models and screens. Identity functions cover registration, login, token refresh, logout, profile editing, password change and password recovery. Catalogue functions cover products, stores, inventory and sales. Analytical functions cover data upload, forecasting, replenishment, dashboards, reports and insights. Governance functions cover user administration, permissions, audit logs, settings, notifications and recovery.

The system treats authorization as a feature requirement rather than an implementation detail. Each protected route declares the relevant permission and every business query includes the authenticated tenant identifier. This approach supports custom permissions in addition to named roles. An owner can therefore grant an analyst forecast access without implicitly granting administration or user-suspension privileges.

Forecast requirements specify both calculation and persistence. A successful response must contain the requested number of dated daily predictions, candidate metrics, a selected model, demand total and

revenue estimate. A response that returns only a total does not satisfy the requirement because it cannot support a daily chart, a lead-time sum or later verification of the horizon.

Table 10: Functional groups and acceptance conditions

Group	Key functions	Acceptance condition
Identity	Register, login, refresh, reset, profile	Invalid credentials fail and revoked sessions cannot refresh
Catalogue	Products, stores, inventory, sales	CRUD is tenant-scoped and validated
Upload	Validate and import CSV	Limits, schema, duplicates and rollback are enforced
Forecast	Run candidates and save results	Horizon contains exactly 7, 14, 28 or 30 dated points
Replenishment	Calculate and manage actions	Quantities are explainable and duplicate open actions are blocked
Analytics	Dashboard, reports and insights	Figures are derived from authorized stored records
Administration	Users, roles, activity	Restricted actions require explicit permissions
Operations	Health, notification, backup	Failures are visible and recovery is verified

3.10 Non-Functional Requirements

Performance requirements constrain interactive waiting time rather than model training alone. Login and common authenticated requests should complete within three seconds under the stated workload, dashboard loading within five seconds and an ordinary forecast within ten seconds. A production load test must include one hundred authenticated users because anonymous health requests do not exercise token validation or tenant queries.

Reliability requirements include controlled errors and recovery. Validation failures return structured client errors, dependency failures return a service-oriented response and unexpected exceptions pass through central JSON error middleware. Daily encrypted snapshots and isolated restore drills provide evidence that stored data can be read after backup rather than assuming that a successful upload equals a usable backup.

Usability requires responsive layouts, visible validation feedback and understandable business terminology. The web client supports wide analytical views, while mobile screens use compact cards and scrollable detail. The same concepts and status labels appear on both platforms. A prepared fifteen-minute user guide and demonstration workspace support first-time use during assessment.

Maintainability depends on separation of responsibilities, configuration through environment variables, reusable middleware and automated checks. Extensibility is supported by a candidate interface that can accept another forecasting model without changing the client request shape. MongoDB indexes and tenant keys support increasing data volume while preserving isolation.

3.11 Principal Use Cases

Table 11: Principal use-case catalogue

ID	Use case	Actor	Precondition	Postcondition
UC-01	Authenticate user	Visitor	Valid credentials	A protected session and profile are returned
UC-02	Create product	Authorized staff	Business workspace exists	Validated product is stored in the tenant
UC-03	Upload sales	Manager or analyst	CSV matches required format	Rows are imported atomically and upload history is saved
UC-04	Run forecast	Forecast operator	Sufficient series history	Candidate metrics and daily predictions are saved
UC-05	Review recommendation	Manager	Forecast and inventory are available	Action is approved, dismissed or completed
UC-06	Generate report	Authorized user	Relevant data exists	Report record and downloadable content are created
UC-07	Invite team member	Owner or administrator	Seat and permission policy allows invitation	Invitation and audit event are stored
UC-08	Restore backup	System operator	Encrypted artifact and key are available	Collections are restored and verified in isolation

3.12 Detailed Forecast Use Case

1. The authenticated user selects a product, store and supported horizon.
2. The client submits the identifiers and horizon to the protected forecast route.
3. Middleware validates the access token, tenant membership and forecasts.run permission.
4. The route loads the product, store, recent sales and inventory within the same tenant.
5. The forecasting service creates a continuous daily history and determines candidate eligibility.
6. Python candidates receive the same history and holdout window; XGBoost is used only when mappings are valid.
7. Each successful candidate returns MAE, RMSE and a daily prediction sequence.
8. The selector chooses the lowest MAE and then the lowest RMSE if required.
9. Revenue and replenishment services calculate totals, stock coverage and risk.
10. The forecast, candidate evidence and daily values are committed to MongoDB.
11. A recommendation and notification are created when the calculated policy requires action.
12. The API returns the saved result and the client updates its chart and summary cards.

3.13 Interface Design Principles

The interface follows a persistent navigation model. Primary functions appear in the sidebar on desktop and in mobile navigation suited to the smaller viewport. Page titles, summaries and action buttons remain consistent so that users can predict where to find a forecast, report or setting. High-risk states use colour and text together; meaning does not depend on colour alone.

Forms validate required fields before submission and preserve server feedback after submission. Destructive actions require an explicit user gesture and authorization. Loading states distinguish an active request from an empty result. Error boundaries provide a recovery screen when an unexpected client rendering error occurs, preventing the user from being left with an unexplained blank page.

Charts supplement rather than replace numeric values. Forecast pages display daily dates, quantities, model identity and metrics in accessible text around the visualization. Product and recommendation tables retain action controls, but mobile screens convert wide records into cards so that critical information remains readable without horizontal compression.

3.14 Data Ownership and Boundary Rules

Business is the tenancy boundary. Products, stores, sales, uploads, forecasts, recommendations, reports, notifications, settings and audit events carry a business reference. The authenticated membership supplies the expected business identifier; clients cannot choose an arbitrary tenant in the request body. Administrative queries are constrained by the same rule.

User identity may exist independently of one business, but membership determines role, status and permissions in the current workspace. Suspension blocks access without deleting historical audit attribution. Invitations are time-limited records and do not grant permissions until accepted. These rules prevent the user model from becoming a shortcut around tenant isolation.

Forecast records preserve references to the product and store together with denormalized identifiers needed for reporting. Deleting a product is restricted or handled deliberately so that

historical forecasts do not silently lose context. Upload records retain counts and validation summaries, while raw temporary files are not treated as permanent business data.

3.15 Feasibility Analysis

Table 12: Feasibility assessment

Dimension	Assessment
Technical	MERN, Python ML, MongoDB Atlas and Expo are compatible with the required cross-platform architecture.
Operational	CSV ingestion matches common SME exports and recommendation explanations remain reviewable by a manager.
Economic	Open-source frameworks and free or entry cloud tiers support an academic prototype without enterprise licensing.
Schedule	Incremental phases allowed authentication, forecasting, replenishment, mobile and operations to be verified separately.
Legal and ethical	Tenant isolation, limited audit metadata and user-controlled actions reduce privacy and automation risk.

3.16 Risk Analysis

Table 13: Principal project risks and mitigations

Risk	Impact	Mitigation
Poor sales quality	Misleading forecast	Strict schema validation, explicit corrections and quality guidance
Short product history	Unstable candidate metrics	Eligibility checks and conservative seasonal fallback
Python service unavailable	Reduced comparison set	Candidate-level failure handling and Node production paths
Credential exposure	Unauthorized access	Environment secrets, hashing, short access tokens and session revocation
Cross-tenant query	Privacy breach	Business filters, ownership checks and tenant-isolation tests
Duplicate import	Inflated demand	Business keys, duplicate validation and idempotent upload handling
Model artifact mismatch	Incorrect inference	Versioned manifest and SHA-256 verification
Backup corruption	Failed recovery	Authenticated encryption, hash checks and isolated restore drills
Mobile network loss	Interrupted operation	Clear retry states and preserved authenticated session where safe
Scope expansion	Late or incomplete delivery	SRS traceability and phased acceptance criteria

3.17 Chapter Summary

The proposed framework combines a tenant-isolated business application with a candidate forecasting service and a distinct replenishment layer. Functional, non-functional, use-case, interface, data-boundary, feasibility and risk analysis show how the framework addresses the problem identified in Chapter 1. Chapter 4 describes the methodology and implementation used to realize and verify this design.

CHAPTER 4
METHODOLOGY AND IMPLEMENTATION

4.1 Development and Research Method

The project followed an iterative engineering method. Requirements were extracted from the approved proposal and SRS, converted into implementation phases, and verified against working endpoints and interfaces. Forecasting work proceeded from data profiling to feature generation, baseline evaluation, recursive backtesting, production model packaging and application integration. Application work proceeded from authentication and database schemas to business workflows, mobile parity, operational controls and final evidence.

Each phase produced a reviewable artifact: a schema, endpoint, interface screen, model bundle, automated assertion or production result. This reduced the risk of completing attractive interfaces without functioning backend behaviour. The final compliance matrix maps all eighteen functional requirements to implementation evidence.

4.2 Dataset and Feature Dataset

The model training dataset contains daily retail sales observations covering 9 August 2017 to 31 October 2019. Feature generation produced 8,924,278 rows across 142 stores and 414 products. Categorical identifiers were converted through stored mappings so the same codes can be used when the serialized model is loaded for inference.

The final production training matrix used 8,172,438 rows after the historical windows needed for lag and rolling variables were available. The trained global XGBoost bundle contains the model JSON, category mappings, registry data, saved future forecasts and a manifest with the exact XGBoost version and SHA-256 checksum.

4.3 Data Validation and Preprocessing

The upload service accepts CSV files up to 10 MB and limits a single import to 10,000 rows. Required headers are checked before processing. Dates are parsed into a standard representation, quantity and revenue fields are validated as numbers, duplicates are detected, and product/store references are resolved within the authenticated workspace. The import is idempotent and uses rollback behaviour so a failed batch does not leave a partial dataset.

For forecasting, sales are aggregated into a continuous daily series. Missing calendar dates are represented with zero demand where appropriate rather than being silently removed. This preserves the real interval between observations and prevents a seven-day lag from meaning seven recorded rows of an irregular series.

4.4 Feature Engineering

Table 14: Feature engineering variables

Feature	Purpose
store_code	Encoded store identity for the global model
product_code	Encoded product identity for the global model
lag_1	Demand from the previous day
lag_7	Demand from the same weekday one week earlier
lag_14	Demand two weeks earlier

Feature	Purpose
lag_28	Four-week seasonal reference
rolling_7	Short-term average demand
rolling_28	Longer recent baseline
day_sin / day_cos	Cyclical weekday representation without an artificial boundary
month_sin / month_cos	Cyclical annual position

Recursive multi-day forecasting feeds each generated prediction into the future history used to construct the next day's lag and rolling values. This is stricter than one-step evaluation because errors can propagate across the horizon, which matches how a real 30-day forecast is consumed.

4.5 Forecasting Algorithms

4.5.1 Linear Regression

The Linear Regression candidate uses engineered lag, rolling and calendar variables to fit a continuous demand target. It provides a fast baseline and helps detect when a complex model adds little value. Negative predictions are clipped to zero because product demand cannot be negative.

4.5.2 ARIMA

The ARIMA candidate models the product/store series directly. The service validates that sufficient observations exist, fits a controlled order, generates the requested horizon and reports a candidate-level error if estimation is not possible. A failed candidate does not abort the entire forecast request.

4.5.3 Random Forest

The Random Forest candidate uses the same engineered supervised-learning matrix as Linear Regression. Multiple decision trees model nonlinear thresholds and interactions, while ensemble averaging reduces the sensitivity of a single tree. A fixed random seed supports reproducible evaluation.

4.5.4 Global XGBoost and Seasonal Fallback

The global XGBoost model uses shared information across mapped stores and products. It was trained with 83 boosting rounds on GPU using XGBoost 3.2.0 and saved as a portable JSON model with a SHA-256 checksum. At production startup, the runtime verifies the manifest and model hash before inference. Series outside the mapping can use the seasonal-naive fallback and the Python candidate service.

4.6 Validation and Model Selection

The validation set follows the training period in time. MAE is computed as $(1/n)\ \Sigma|y_i - \hat{y}_i|$. RMSE is computed as $\sqrt{[(1/n)\ \Sigma(y_i - \hat{y}_i)^2]}$. The candidate with the smallest MAE is selected; RMSE is used when MAE values are tied. Candidate metadata is retained for transparency, and any invalid or failed model is excluded with its error recorded.

For the imported production model, recursive backtests used cutoffs of 9 August 2019, 6 September 2019 and 4 October 2019. Each fold trained only on earlier rows and evaluated the following 28 days. The combined result contains 986,433 observations.

4.7 Forecast Request Pipeline

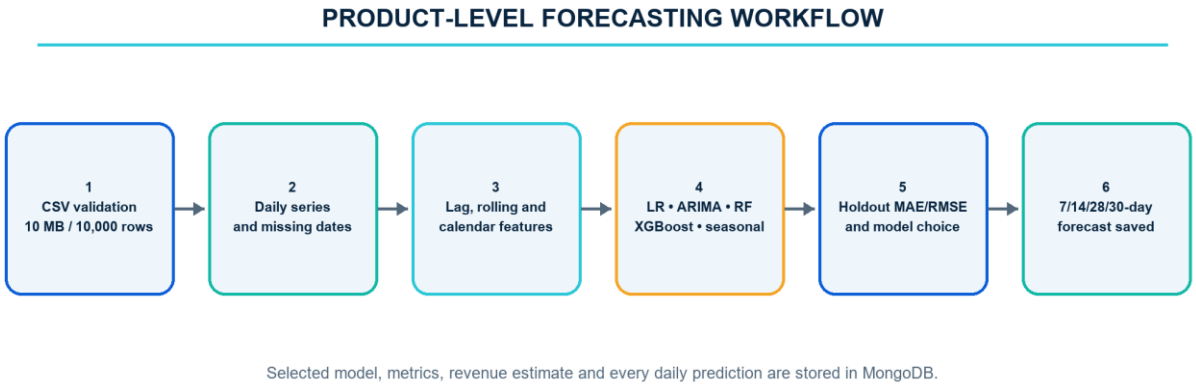


Figure 2: Product-level forecasting workflow

A successful run stores the horizon, selected model, MAE/RMSE, prediction dates and quantities, forecast units, forecast revenue and inventory plan. Storing all daily points is necessary for charts, reports, later comparison and the verified requirement that a 30-day run contains thirty predictions.

4.8 Revenue and Replenishment Calculations

Forecast revenue uses historical average unit revenue where available; otherwise the current product price is used. For inventory planning, average daily demand equals total forecast units divided by horizon days. Lead-time demand equals average daily demand multiplied by supplier lead-time days. Demand variability is the population standard deviation of historical daily quantities.

Variability safety stock equals 1.65 × demand standard deviation × √lead time. Policy safety stock equals lead-time demand × configured safety-stock percentage. The system uses the larger value. Reorder point is the ceiling of lead-time demand plus safety stock. Target stock is the ceiling of horizon forecast units plus safety stock. Recommended order quantity is max(0, target stock − available stock), where available stock is on-hand stock less reserved stock.

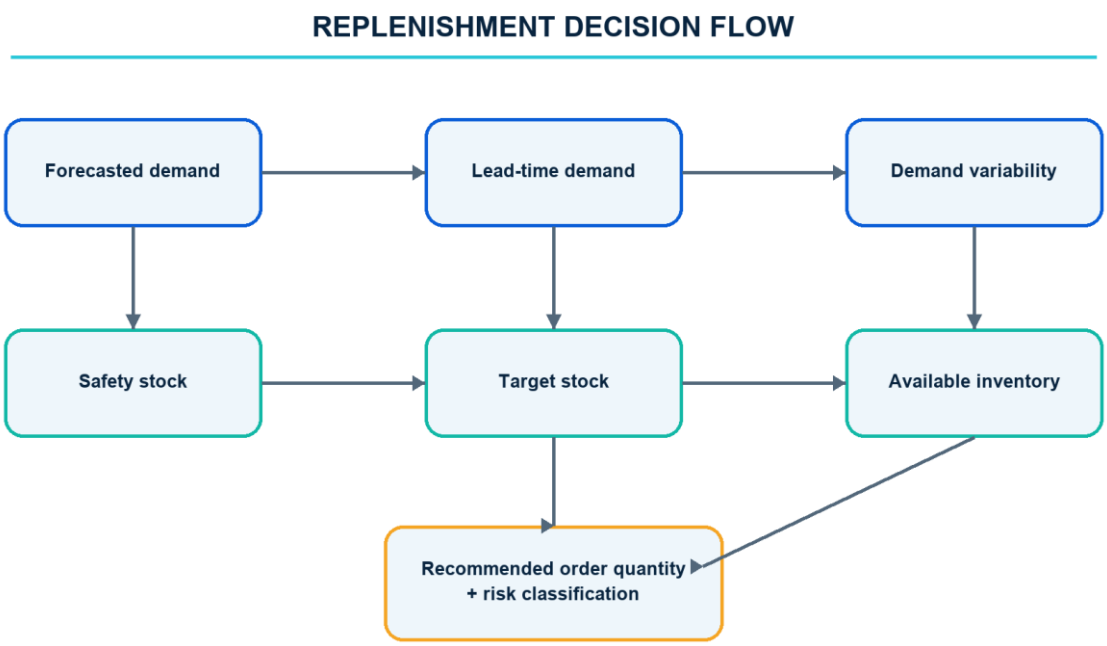


Figure 3: Replenishment decision flow

An order is high risk when available stock is zero or provides two or fewer forecast days of cover. Overstock is flagged when available inventory exceeds 125 percent of target stock. A demand spike requires forecast average demand to be at least 25 percent above the historical mean and at least one historical standard deviation higher. Duplicate open recommendations for the same business, product, store and type are prevented.

4.9 Software Implementation

Table 15: Technology stack

Layer	Technology	Implementation responsibility
Web	React, Vite, Recharts	Responsive dashboard, products, upload, forecast, reports, insights, admin and settings
Mobile	React Native, Expo	Secure cross-platform access to the same business functions
API	Node.js, Express, Zod	REST routes, validation, sessions, permissions, orchestration and errors
Persistence	MongoDB Atlas, Mongoose	Tenant business records, indexes and TTL audit retention
Python ML	Pandas, scikit-learn, Statsmodels	LR, RF and ARIMA candidate forecasts
Production ML	XGBoost JSON runtime	Verified global recursive inference and seasonal fallback
Operations	Vercel, GitHub Actions, EAS	Deployment, monitoring, backup verification and mobile builds

4.10 Security Implementation

Passwords are hashed with bcrypt and never returned by the API. Access tokens are short lived. Refresh sessions are revocable and include a session version. The web client receives its refresh credential through an HttpOnly cookie, reducing exposure to client-side scripts. The mobile client uses Expo SecureStore. Logout revokes the refresh session, and password change or reset invalidates prior sessions.

Route middleware checks authentication, named permissions and tenant membership. Every business-scoped model query includes the authenticated business identifier. Zod schemas validate JSON inputs, Multer limits file size, and the central error middleware returns controlled JSON

messages. Audit events record logins, uploads, forecasts, profile/security changes and administrative actions with a one-year retention period.

4.11 Notifications and Reports

Persistent in-app notifications are created when an upload completes, a forecast becomes ready, stock becomes low or critical, or a high-risk recommendation is generated. Expo device registrations enable mobile push delivery when platform credentials are present. SMTP delivery is optional and uses the same event pipeline, so the application remains functional when outbound email is not configured.

Users can generate sales, inventory, forecast and replenishment reports. Report records remain tenant-scoped and can be downloaded by authorized web and mobile users. Replenishment reports include recommended quantities, risk, safety stock and status so purchasing actions can be reviewed outside the live dashboard.

4.12 Backup, Monitoring and Recovery

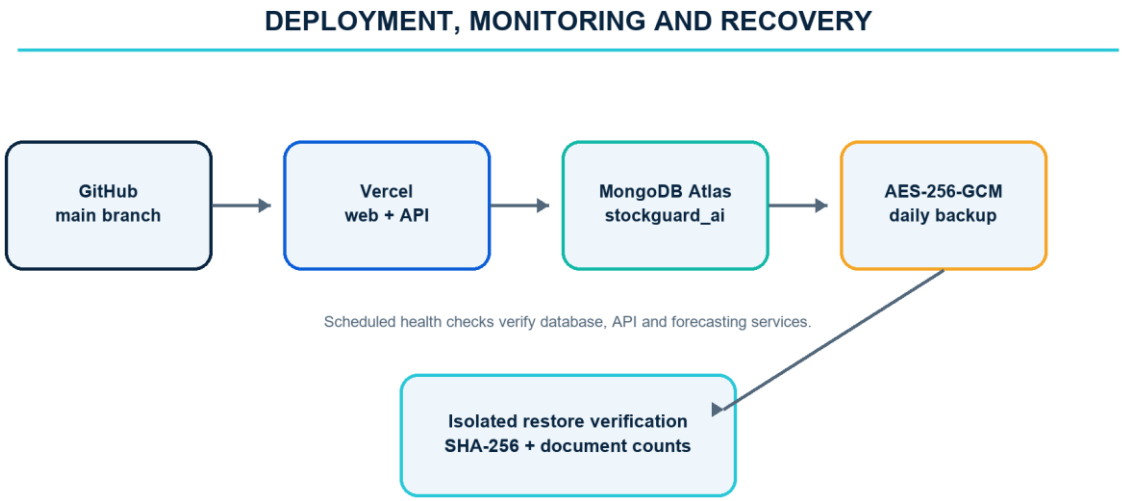


Figure 4: Deployment, monitoring and recovery workflow

A scheduled GitHub Actions workflow exports the MongoDB database, packages collection data with a manifest, encrypts it using AES-256-GCM and retains the artifact for thirty days. Restore verification decrypts the artifact, validates its hash and restores records to temporary collection names. It compares document counts and removes the temporary collections after verification, avoiding interference with live business data.

A separate scheduled health workflow checks the production API, MongoDB connectivity, Python ML status, XGBoost availability and supported horizons. These checks create operational evidence and provide early warning when an external service is unavailable.

4.13 Testing Strategy

Verification was organised at four levels. Static checks inspect route, model and permission definitions. Integration checks create isolated users and business records to test authentication, products, imports, forecasts, analytics and tenant isolation. Client builds validate web bundling and mobile TypeScript/export compatibility. Production checks exercise deployed login, dashboard, CSV validation, forecasting and concurrency thresholds.

Test data is isolated from normal business records and cleaned after verification. The dedicated presentation workspace is deterministic so the defence demonstration does not depend on last-minute data entry. Backup restoration also uses isolated collection prefixes, which allows integrity checks without overwriting production collections.

4.14 Chapter Summary

The implemented methodology connects validated sales ingestion, continuous daily-series construction, shared feature engineering, time-ordered model evaluation and recursive multi-day forecasting. The same forecast result feeds revenue estimation and the replenishment engine, ensuring that dashboard values, reports, notifications and recommended quantities are derived from one stored analytical record.

The software layers mirror this workflow: React and React Native provide user access, Express enforces business rules and security, Python and the XGBoost runtime provide forecasting candidates, MongoDB stores tenant-scoped records, and scheduled workflows verify health and recovery. Chapter 5 evaluates the resulting model and application against recorded evidence.

4.15 Iterative Development Process

Development followed an incremental process organized around demonstrable vertical slices. The first slice established authentication, tenant creation and the base web interface. The second introduced products, stores, sales and validated CSV ingestion. Forecasting was then connected from the client through the Node API to Python and production-model candidates. Replenishment, reports and notifications were built on the saved forecast record rather than on temporary client calculations.

Mobile implementation reused the established API after the server behaviour had integration evidence. Administration, permissions and audit activity were subsequently added around the same tenant model. Operational work completed the path with production health checks, encrypted backup, recovery verification and performance measurement. This order reduced uncertainty because every later feature depended on a tested domain object from an earlier phase.

Each phase used a definition of done that combined code, persistent data and visible behaviour. A forecast phase, for example, was not complete merely because a function returned an array; the API had to save the exact horizon, the clients had to display it and an integration check had to retrieve all daily values. This evidence-driven definition prevented the project from appearing complete only at the interface level.

4.16 CSV Ingestion Algorithm

1. Reject files larger than 10 MB before parsing.
2. Read the header and normalize harmless surrounding whitespace without changing field meaning.
3. Require the submitted column set and reject unknown structures that cannot be mapped safely.
4. Parse each date with an explicit accepted format and reject impossible dates.
5. Parse quantities and revenue as finite numeric values and apply domain constraints.
6. Resolve product and store identifiers inside the authenticated tenant.
7. Detect duplicate business keys inside the file and against the selected idempotency policy.
8. Stop before commit when any blocking error exists and return row-level feedback.

9. Insert accepted records in a controlled transaction-like workflow and roll back on failure.
10. Store upload counts, status, actor and timestamps, then create an upload-completed notification.

Validation and import are separated so the user can inspect errors before records change. This is particularly important for a large file because partial success can be difficult to reconcile with the source spreadsheet. The API also limits row count to 10,000, protecting memory and request duration while satisfying the SRS. Larger historical datasets can be split into controlled batches.

Idempotency prevents accidental double counting when the same upload is retried after a network interruption. The implementation checks stable business identifiers and preserves an upload record. Duplicate handling is explicit rather than relying on the user to notice an unexpectedly high dashboard total after import.

4.17 Forecasting Procedure

For a requested product and store, the service retrieves chronologically ordered daily quantities and builds a complete date index. Candidate evaluation uses a recent holdout whose size is compatible with the history and requested horizon. The training portion is never allowed to include holdout targets. Each candidate returns predictions aligned with the same actual dates so MAE and RMSE are comparable.

Linear Regression receives time index, lag, rolling and cyclical calendar variables. ARIMA operates on the ordered univariate quantity series and selects a conservative order suitable for the available sample. Random Forest uses the engineered supervised rows and recursive future construction. The global XGBoost booster uses its trained twelve-column feature order and mapping files. Seasonal naive repeats the most relevant previous seasonal observation and requires minimal estimation.

For future day t, lag values are drawn from the observed history plus predictions already generated for earlier future days. Rolling values use only entries available before t. Calendar values are calculated directly from the future date. Negative predictions are clipped to zero because negative product demand is not meaningful in the selected domain. The system retains numeric precision for calculation and applies display rounding separately.

Candidate selection first removes failed or ineligible results. The lowest holdout MAE wins; RMSE resolves a tie and a deterministic candidate order resolves any remaining equality. The selected model is then refitted or executed on the full available history to produce the requested future horizon. The final record contains both comparison evidence and production predictions.

4.18 Replenishment Formula Implementation

Let d-hat sub t denote forecast demand for future day t and L denote supplier lead time in days. Lead-time demand is the sum of predictions from day one through the smaller of L and the forecast horizon. Historical variability is estimated from recent daily demand. Safety stock is calculated from variability, the square root of lead time and the configured safety factor, preserving the familiar relationship that uncertainty grows more slowly than time under independent variation.

Available stock equals on-hand stock minus reserved stock, with a lower bound of zero. Target stock adds lead-time demand, safety stock and the demand expected during the configured review period. Recommended order quantity is max(0, target stock minus available stock). Days of cover

divides available stock by a guarded estimate of average future daily demand. A zero-demand series produces a bounded result rather than a division error.

Risk rules interpret the calculated quantities. Critical status applies when available inventory cannot cover the earliest high-risk demand window or when days of cover is extremely low. Low stock applies when stock is below the reorder point but not critical. Overstock applies when coverage materially exceeds the target policy. Demand spike applies when near-future average demand exceeds the historical baseline by the configured threshold.

A unique open-recommendation rule combines tenant, product, store and recommendation type. When the same unresolved risk is recalculated, the existing record is updated rather than creating a second open action. Status changes record the actor and time. Completed or dismissed records remain available for audit while a later materially new risk can open a new recommendation.

4.19 REST API Design

Routes use resource-oriented paths and JSON representations. Authentication routes issue and rotate sessions. Domain routes validate identifiers and body data before service execution. Success responses return the persisted resource or an explicit summary, while client errors identify invalid fields and service errors preserve a stable public message. Central middleware prevents raw stack traces from becoming an API response.

Permission checks occur before expensive database or model work. Ownership checks follow when a specific resource identifier is supplied. Pagination and filters are applied to list routes to control response size. Dashboard routes use database aggregation rather than transferring all sales to the browser. These choices improve security and performance together.

The Node service calls the Python model endpoint with a bounded request and timeout. Candidate errors are normalized into the comparison structure. Health output distinguishes the public API, database, Python service and XGBoost readiness, allowing operations to diagnose partial availability. The user-facing forecast route still applies policy and persistence in Node so the specialist service does not become a second uncontrolled business API.

4.20 Web Application Implementation

The React web application organizes authenticated pages under a common layout with sidebar navigation, search, status indicators and profile controls. Route guards wait for session restoration before deciding whether to display the requested screen. This prevents a valid returning user from briefly seeing a sign-in redirect while a refresh request is still active.

Server state is loaded through a shared API client that attaches access credentials and handles refresh. Pages provide loading, empty, error and success states. Product forms support create and update operations; forecast pages preserve the selected series and horizon; settings controls persist to the server rather than remaining visual-only toggles. Download actions use authorized report endpoints.

Responsive breakpoints reorganize analytical grids for tablets and narrow screens. Tables preserve critical columns and provide card alternatives where required. The supplied StockGuard logo is used as a contained brand mark rather than stretched across navigation. Dark and light appearance settings use shared tokens so contrast remains consistent across screens.

4.21 Mobile Application Implementation

The Expo React Native application implements login, dashboard, products, CSV validation, forecasting, reports, insights, profile, settings, notifications, administration and replenishment views. It communicates with the production API and does not maintain a separate mock database. Navigation state reflects the signed-in session and role permissions.

Refresh credentials are held in Expo SecureStore. Access tokens remain short-lived and are renewed through the same session mechanism as the web application. Push registration associates the device token with the authenticated user and business. Notification selection can navigate to the relevant forecast, product or recommendation when the payload contains a supported target.

Mobile screens use touch-sized controls and compact cards. Wide model-comparison tables are represented as stacked candidate cards with MAE, RMSE and selection state. Product edits and recommendation status actions call the same API routes as the web client. TypeScript checks and an Expo Android production export provide build evidence; the codebase remains compatible with iOS through Expo, subject to Apple distribution credentials.

4.22 MongoDB Persistence and Indexing

Mongoose schemas define required fields, defaults, references and timestamps. Compound indexes begin with business where queries are tenant-scoped. Product SKU and store code uniqueness is enforced within a business rather than globally. Sales indexes support product-store-date history retrieval, while forecast and recommendation indexes support recent lists and open-risk checks.

Documents are appropriate because forecast candidates, daily prediction arrays and notification metadata have bounded nested structures that are commonly read together. References are retained for shared entities such as user, product, store and business. Aggregation pipelines join or group only the fields required for dashboard metrics, reducing memory and network transfer.

TTL indexes remove time-limited reset tokens, invitations or audit records according to policy. Deletion operations consider references and audit needs. Production data changes pass through service functions so inventory synchronization, recommendation recalculation and activity logging occur consistently.

4.23 Authentication and Authorization Procedure

1. Normalize and validate submitted email and password fields.
2. Look up the user and active business membership without exposing which field failed.
3. Compare the supplied password with the bcrypt hash.
4. Issue a short-lived signed access token containing identity and session claims.
5. Create or rotate a refresh session bound to the current session version.
6. Return the permitted profile and store the web refresh value in an HttpOnly cookie or mobile secure storage.
7. On protected requests, verify signature, expiry, session version, active status and membership.
8. Resolve permissions from role defaults and authorized custom overrides.
9. Apply the tenant identifier to resource queries and record selected security events.
10. On logout, reset or password change, revoke the relevant session records.

4.24 Notification Delivery

Domain services create a persistent notification first. This guarantees that the user can see the alert in the application even if an optional external channel fails. The record includes type, severity, title, message, read state, business, recipient and target metadata. Upload completion, forecast readiness, low stock, critical inventory and high-risk recommendation events use this shared structure.

Email delivery is attempted when SMTP configuration and user preferences permit it. Mobile push delivery is attempted when the recipient has registered Expo tokens and enabled the relevant category. Failures are recorded for operations but do not roll back the completed upload or forecast. This ordering prevents communication infrastructure from corrupting the primary business transaction.

4.25 Deployment and Continuous Verification

The repository contains Vercel configuration for the web and API deployment, an entry point for the Python forecasting service and EAS profiles for mobile builds. Environment variables separate database, token, email, push, model and backup credentials from source control. The production URL uses HTTPS, and MongoDB Atlas network and user controls restrict database access.

GitHub Actions schedules production health checks and encrypted backups. The health workflow confirms application availability and records dependency state. The backup workflow exports the StockGuard database, encrypts the artifact with AES-256-GCM, retains it for the configured period and performs an isolated restore verification. Temporary restored collections are removed after counts and readability are checked.

Verification commands cover server static assertions, authenticated database integration, forecast persistence, analytics isolation, web production build and mobile TypeScript/export checks. Running the checks after deployment reduces the gap between local correctness and production behaviour.

4.26 Ethical Considerations

Forecasts may influence purchasing decisions, but the system does not represent them as certainty. Metrics, model identity and assumptions remain visible, and recommendations require a user-controlled status change. This human review is important because the dataset may omit promotions, supplier disruption or local knowledge.

The project minimizes sensitive data by focusing on business transactions and account information required for operation. Passwords, tokens and backup keys are not logged. Tenant isolation and permissions reduce unintended disclosure among businesses and employees. Test tenants are separated from presentation data and are removed after verification where the script supports cleanup.

Reported metrics distinguish measured evidence from interpretation. XGBoost results come from recursive backtesting, while live candidate scores depend on the selected series. Performance values identify the test workload and environment. External services such as SMTP, app-store publication and long-term uptime are described as configured capabilities or operational dependencies rather than claimed as evidence when credentials or elapsed observation time are unavailable.

4.27 Chapter Summary

The implementation follows an evidence-driven, layered methodology. Validated data enters a tenant-scoped persistence layer, forecasting candidates are compared chronologically, replenishment

rules turn predictions into actions, and web/mobile clients consume one protected API. Security, notifications, deployment, backup and testing are integrated into the same design. Chapter 5 evaluates the resulting model and application behaviour.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Model Training and Backtest Results

The imported model bundle records 8,172,438 final training rows, 83 boosting rounds, XGBoost 3.2.0 and a 3.44 MB compressed bundle. The feature dataset covered 142 stores, 414 products and dates from 9 August 2017 to 31 October 2019. The saved manifest records the model SHA-256 value 61dda9da18aa090ed26304319c7162215f0a0158a46af18dbfcc555f92851c3c.

Table 16: Three-fold recursive backtest results

Fold	Cutoff	Observations	XGBoost MAE	XGBoost RMSE	Seasonal MAE	Seasonal RMSE
1	2019-08-09	340,765	0.588152	2.540696	0.674038	2.869917
2	2019-09-06	324,889	0.559697	1.476403	0.620590	1.852476
3	2019-10-04	320,779	0.576340	1.872189	0.681093	2.525994
Combined	Three folds	986,433	0.574939	2.021802	0.658729	2.459768

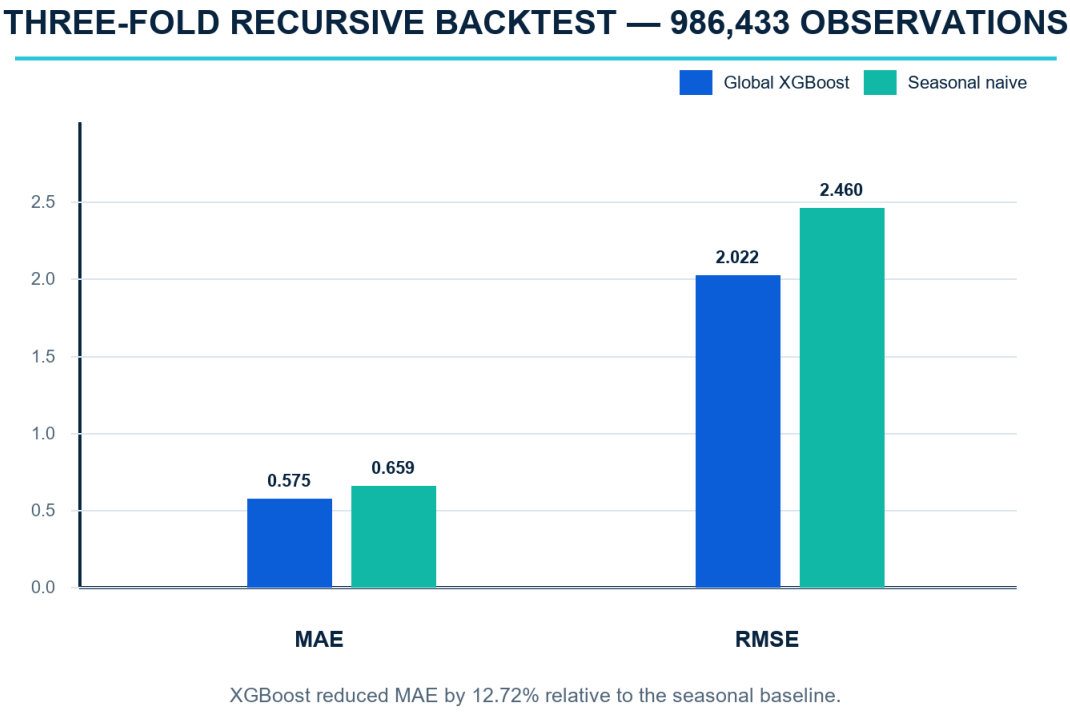


Figure 5: Three-fold recursive XGBoost backtest

The 12.72 percent MAE improvement shows that the global XGBoost model provided a more accurate central forecast than repeating the seasonal value across the tested series. RMSE also improved, indicating fewer or smaller large errors overall. Fold 1 has higher RMSE than the later folds, demonstrating why a single validation window would give an incomplete view of performance.

5.2 Live Candidate Comparison

The live application sends the selected product/store history to the Python service and obtains Linear Regression, ARIMA and Random Forest candidates with MAE and RMSE. In parallel, the Node runtime evaluates XGBoost where identifiers map to the training registry and provides a seasonal candidate. The API normalizes the candidate objects, removes failures and chooses the lowest MAE with RMSE as the tie-breaker.

This design satisfies the proposal's required comparison while preserving the separately trained production model. It also avoids claiming that one algorithm is always best. A linear or seasonal model can win for a stable series, while a tree ensemble can win for a nonlinear or irregular series. The selected model is stored with each run so the decision remains inspectable.

5.3 Functional Verification

Table 17: Functional verification summary

Area	Verified result
Authentication	Registration, login, refresh, logout, revocation and password reset passed
Account	Profile persistence, audited password change and old-session invalidation passed
Products/imports	Tenant CRUD, 10 MB CSV checks, idempotency and rollback passed
Forecasting	Python comparison, XGBoost/seasonal candidates and thirty saved daily points passed
Replenishment	Safety stock, order quantity, lifecycle and high-risk alerts passed
Analytics	Date windows, revenue, categories, low stock and tenant isolation passed
Web	Vite production build passed
Mobile	TypeScript validation and Expo Android production export passed
Dependencies	Production server audit recorded zero known vulnerabilities at verification time
Recovery	Encrypted backup codec, tamper detection and isolated restore passed

5.4 Performance Evaluation

The production performance test used 100 authenticated virtual users over a ten-second ramp. Every request returned HTTP 200. The average response time was 259 ms, p95 was 832 ms and the maximum was 1,210 ms. Login completed in 855 ms, the dashboard in 843 ms, a live 30-day forecast in 2,224 ms and validation of a 10,000-row CSV fixture in 61 ms.

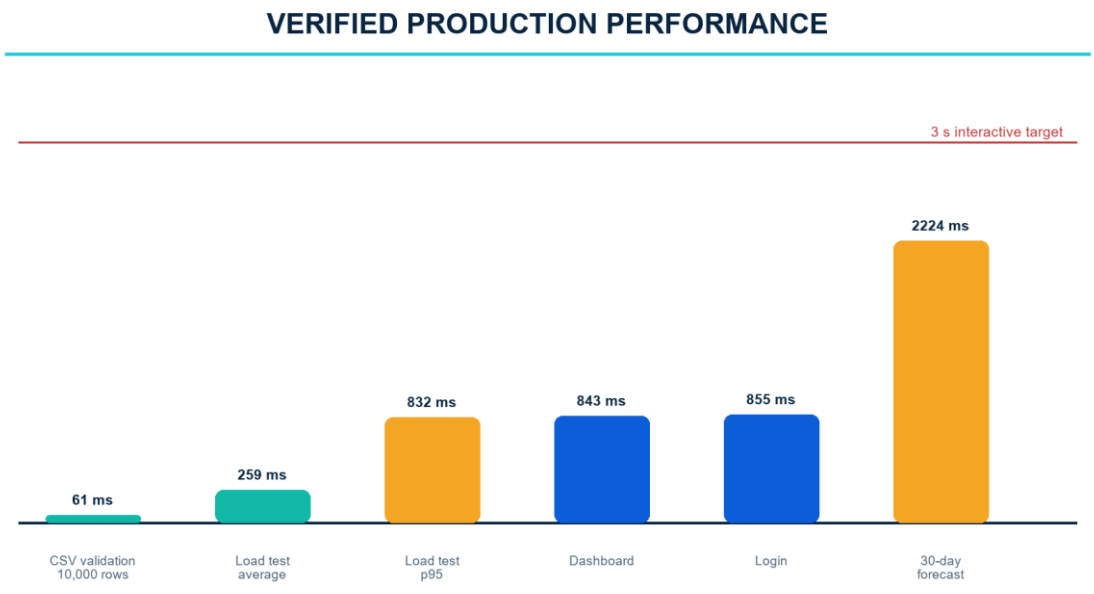


Figure 6: Verified production performance

The measured values satisfy the SRS interactive targets: login and p95 concurrent requests remained below three seconds, dashboard loading remained below five seconds, and forecast generation remained below ten seconds. The result also satisfies the requirement to support 100 concurrent users without errors. Cold-start measurements were higher than warm requests but remained within the broader operational limits.

5.5 Recovery Verification

The stored machine-readable recovery evidence shows a successful local isolated drill that restored 1,055 documents across 16 collections in 13.914 seconds and removed the prefixed temporary collections after verification. A later scheduled production run recorded in the QA report restored 1,137 documents across 16 collections in 23.454 seconds. Both results are well within the ten-minute recovery objective.

Encryption and hash verification protect confidentiality and detect modified backup content. Because restoration uses temporary collection prefixes, the workflow verifies readability and document counts without replacing the live database. This makes the recovery check safe to run as an operational control rather than leaving recovery as an untested assumption.

5.6 Presentation Workspace

The production demonstration workspace contains eight products, 800 sales records, a complete 30-day forecast, three downloadable reports, fourteen notifications and four replenishment recommendations. This provides a stable end-to-end path for the internal defence: sign in, review the dashboard, inspect products, upload or validate data, run a forecast, examine the selected model, review an order recommendation and open the same workspace on mobile.

5.7 Requirements Compliance

All functional requirements REQ-1 through REQ-18 have implementation evidence. Performance requirements were validated with timed production checks. Security requirements are covered by password hashing, JWT sessions, permission middleware, tenant filters and HTTPS. Reliability requirements are supported by controlled errors, health monitoring and verified recovery. Usability is supported by responsive navigation, explicit validation messages and a fifteen-minute user guide.

5.8 Discussion

The strongest project result is the connection between predictive modelling and an operational decision. A stored 30-day sequence supports graphs and reports, but the replenishment calculation makes the output directly useful. The recommendation lifecycle also recognises that a model advises rather than automatically purchases; a business user remains responsible for approval and can dismiss a recommendation when contextual information is missing from the data.

The hybrid model architecture is another important result. The Python service provides the three algorithms promised in the proposal. The imported XGBoost bundle adds a high-volume global candidate backed by recursive validation, and the seasonal model provides resilience for short or unmapped series. Candidate-level failure handling prevents one unavailable algorithm from crashing the entire workflow.

Measured performance shows that the application can satisfy its target workload in the deployed environment. The mobile client and web client use the same API, which reduces duplicated business rules. Tenant-scoped queries and auditable permission checks make the SaaS structure defensible beyond the interface layer.

5.9 Limitations

- No forecasting system can compensate for incorrect dates, missing products or sales that do not represent normal operations.
- Promotions, holidays, supplier reliability, weather and economic variables are not yet included as external features.
- The global XGBoost model is strongest for identifiers represented by its training mappings; new series need sufficient local history.
- SMTP email and public app-store delivery depend on credentials and approvals outside the submitted repository.
- Long-term availability statistics require monitoring over a longer production period than the development schedule.

5.10 Fold-Level Interpretation

The first recursive fold used the earliest cut-off and produced the highest XGBoost RMSE. This is reasonable because the training window was shorter and the evaluated period may contain demand spikes that amplify squared error. Even in this fold, XGBoost MAE remained below the seasonal baseline, indicating that the improvement was not created solely by one favourable late period.

The second fold produced the lowest errors for both models. The XGBoost MAE of approximately 0.560 and RMSE of 1.476 show that most daily series observations were predicted close to the actual quantity, with fewer extreme residuals than in the first fold. The seasonal baseline improved as well but retained a larger error, supporting the value of lag, rolling and calendar interactions.

The third fold represents the latest validation window before final production training. XGBoost recorded MAE of approximately 0.576 and RMSE of 1.872, while the seasonal baseline recorded MAE of approximately 0.681 and RMSE of 2.526. The direction of improvement remained stable across all folds. This stability is more informative than a single random split because every fold predicts a future interval from earlier data.

The combined 12.72 percent MAE improvement is an aggregate description, not a guarantee for every product. The production registry evidence showed that some series were better served by seasonal naive. The final application therefore preserves candidate comparison and fallback instead of asserting that the global booster must always be selected.

5.11 Error Metric Interpretation

An overall XGBoost MAE of 0.574939 means that the average absolute deviation across the combined evaluated observations was a little over half a unit. Because retail quantities are discrete but model predictions are continuous, this value is plausible and useful for comparing candidates. It should not be interpreted as every forecast being within one unit; individual high-volume series can have larger residuals.

RMSE is higher than MAE because it penalizes larger errors more strongly. The gap between MAE and RMSE indicates the presence of occasional observations with greater deviation. Inventory policy addresses part of this risk through safety stock rather than pretending that a point forecast is exact. A future extension could store prediction intervals and use their upper bound directly in service-level calculations.

Series-level metrics are displayed with the forecast so a user can identify when a locally fitted model performs poorly despite strong global evidence. The model score is paired with data sufficiency and candidate status. This discourages false precision and supports a manager's decision to gather more history or apply a conservative order policy.

5.12 Thirty-Day Forecast Persistence Result

The thirty-day integration check verifies the requirement at persistence level. The request uses horizon 30, the selected candidate returns thirty future quantities, and MongoDB stores thirty objects with consecutive dates. Retrieval through the saved-forecast endpoint returns the same count and horizon. The test also verifies candidate metrics and summary totals.

This check catches defects that a chart screenshot may miss. A client could interpolate thirty labels from a weekly total, or a server could calculate thirty points but truncate the stored array. Counting the persisted daily predictions establishes that the analytical record itself satisfies the horizon. Replenishment can therefore sum the exact lead-time subset and reports can reproduce the same sequence.

5.13 Replenishment Case Analysis

Table 18: Illustrative interpretation of replenishment outputs

Condition	Calculated evidence	System response	Manager interpretation
Critical stock	Available stock below immediate demand	High-risk recommendation and alert	Review and order immediately
Low stock	Available stock below reorder point	Open replenishment recommendation	Approve within normal purchasing cycle
Adequate	Available stock covers target policy	No positive order quantity	Monitor; no new order required
Overstock	Coverage materially exceeds forecast	Overstock warning	Reduce purchasing or plan promotion
Demand spike	Future rate exceeds recent baseline	Spike recommendation and notification	Check event context and supplier capacity

The recommendation is deliberately advisory. Approval confirms that an authorized user accepts the system evidence for operational follow-up; it does not claim that a purchase order has been transmitted to a supplier. Completed status can be applied when the organization has acted. Dismissed status preserves the record and may include a contextual reason such as an expected promotion ending or a supplier already contacted outside the platform.

Duplicate prevention was verified around open records. Recalculating the same product-store risk updates the evidence instead of creating repeated alerts. Once the previous action is completed or dismissed, a later forecast can create a new open recommendation if the condition returns. This behaviour keeps the dashboard actionable while preserving history.

5.14 Acceptance Test Results

Table 19: Representative acceptance-test results

Test	Expected result	Observed result	Status
Invalid login	Generic rejection without session	401 response and no session	Pass
Refresh rotation	New token; old refresh invalid	Rotation and revocation verified	Pass
Cross-tenant product	Resource remains inaccessible	Ownership filter denied access	Pass
Oversized CSV	Reject before import	10 MB limit enforced	Pass
Duplicate CSV rows	No duplicated sales	Validation/idempotency passed	Pass
Thirty-day forecast	Thirty dated points saved	Count and retrieval verified	Pass
Candidate comparison	MAE/RMSE saved	Metrics stored per candidate	Pass
Recommendation lifecycle	Valid status transitions	Open/approved/dismissed/completed passed	Pass

Test	Expected result	Observed result	Status
Report authorization	Only tenant reports returned	Tenant list and download passed	Pass
Mobile build	Typed app exports	TypeScript and Android export passed	Pass
Concurrent access	100 users without errors	100 HTTP 200 responses	Pass
Encrypted restore	Verified isolated recovery	Collections restored and removed	Pass

5.15 Security Evaluation

Authentication integration checks covered registration, login, refresh, logout, password change, reset and session revocation. A reset token is stored as a hash, expires after the configured window and becomes unusable after completion. Changing credentials increments session state so earlier web and mobile refresh tokens cannot silently create a new access token.

Authorization checks covered named roles and explicit permissions. Administrative functions require administration rights; forecast execution requires its own permission; read and write capabilities are separated where needed. Tenant-isolation tests use resources from a different test business and confirm that a valid token alone does not bypass ownership filters.

Dependency audit evidence recorded zero known production server vulnerabilities at the verification time. This result is time-sensitive and should be repeated because new advisories can appear after submission. HTTPS is provided by the production platform, while database and workflow secrets remain in managed environment settings rather than the repository.

5.16 Performance Analysis

The one-hundred-user test exercises authenticated application behaviour. An average response time of 259 ms demonstrates that common warm requests are fast, while p95 of 832 ms shows that ninety-five percent completed well below the three-second requirement. The maximum of 1,210 ms remained acceptable and there were no failed responses during the measured ramp.

The separate forecast measurement of 2,224 ms includes model coordination and persistence for a thirty-day result. It remains below the ten-second SRS target. CSV validation of 10,000 rows completed in 61 ms in the measured local validation path, indicating that schema checks are not a bottleneck relative to upload transfer and database insertion.

Performance evidence represents the tested deployment, data volume and time. It does not prove unlimited scalability. MongoDB index design, bounded uploads and database-side aggregation support growth, while production monitoring is required to observe behaviour under a longer sustained workload. The report therefore claims compliance with the defined target rather than a universal throughput figure.

5.17 Reliability and Recovery Analysis

The local machine-readable drill and the later scheduled production run both restored sixteen collections in isolation. The document counts differ because the production workspace accumulated additional records between runs. Both executions completed in seconds rather than minutes, leaving substantial margin beneath the ten-minute objective.

Authenticated encryption detects an incorrect key or modified ciphertext before restored data is trusted. Temporary collection prefixes prevent the verification task from overwriting live names. Count and readability checks establish that the archive is structurally usable, after which temporary collections are removed. This is stronger evidence than checking only that an encrypted file exists.

The daily schedule creates a maximum nominal recovery point interval of approximately twenty-four hours for the repository workflow. A commercial deployment could add MongoDB continuous backup and point-in-time recovery to reduce that interval. The prototype's implemented control satisfies the submitted daily-backup requirement and demonstrates the recovery procedure end to end.

5.18 Requirements Traceability Evaluation

Requirements REQ-1 through REQ-18 map to implementation and verification evidence. Identity requirements map to authentication routes and session tests. Data-management requirements map to product, sales and upload services. Forecasting requirements map to Python and Node candidates, metric selection and persisted horizon checks. Business-intelligence requirements map to dashboard, report, insight and replenishment endpoints.

Administration, security and reliability requirements map to permission middleware, tenant filters, user-management routes, audit retention, health checks and encrypted recovery. This traceability reduces ambiguity at defence time: each claim can be demonstrated in the interface, inspected in a response or reproduced by a repository command.

5.19 Threats to Validity

- Historical validity: the Kaggle data ends in 2019 and may not represent every present-day SME or product category.
- Construct validity: MAE and RMSE measure forecast error but do not directly measure profit, service level or user satisfaction.
- External validity: performance on the tested deployment and presentation tenant may differ at much larger scale or in another region.
- Internal validity: candidate comparisons depend on the selected holdout size, preprocessing rules and availability of sufficient history.
- Operational validity: optional email delivery and public mobile-store distribution depend on external credentials and approvals.
- Decision validity: a recommended order quantity assumes that recorded inventory, lead time, reserved stock and price are current.

These limitations do not invalidate the prototype, but they define the boundary of its claims. The system demonstrates a verified forecasting and replenishment workflow; future field studies are required to quantify long-term commercial outcomes across diverse SMEs.

5.20 Chapter Summary

Results show consistent XGBoost improvement over the seasonal baseline across three recursive folds, complete thirty-day persistence, functional compliance across the application, acceptable one-hundred-user performance and verified encrypted recovery. The discussion also shows why candidate-level evidence and human-reviewed recommendations are necessary. Chapter 6 summarizes the contributions and future direction.

CHAPTER 6
SUMMARY, CONTRIBUTIONS AND CONCLUSION

6.1 Summary

StockGuard AI was designed to help SMEs replace manual demand estimates with an integrated forecasting and inventory decision process. The final prototype includes secure identity, multi-user roles, product and sales management, strict CSV ingestion, five forecasting paths, 7/14/28/30-day horizons, revenue estimation, replenishment recommendations, dashboards, reports, insights, notifications, administration and web/mobile access.

The system was deployed and verified against the SRS. Model evidence includes a three-fold recursive XGBoost backtest over 986,433 observations. Application evidence includes authenticated integration checks, a 100-user production load test, complete thirty-point forecast persistence and encrypted isolated recovery.

6.2 Main Findings

1. The global XGBoost model reduced MAE by 12.72 percent relative to the seasonal baseline across the combined recursive backtest.
2. Live per-series comparison can evaluate Linear Regression, ARIMA and Random Forest and can include XGBoost and seasonal candidates when eligible.
3. Forecast totals can be converted into lead-time demand, safety stock, target stock and a concrete order quantity without manual spreadsheet formulas.
4. The deployed API served 100 authenticated virtual users with zero errors and p95 response time of 832 ms.
5. Encrypted backup artifacts can be restored and verified in isolated collections far within the ten-minute recovery objective.

6.3 Contributions

The project contributes a complete SME-oriented forecasting workflow rather than a model demonstration alone. It integrates model comparison with persistent daily forecasts, revenue calculations and replenishment actions. It also contributes a consistent web/mobile experience backed by one tenant-isolated API, reducing behavioural differences across platforms.

From an engineering perspective, the project provides traceability from SRS requirements to routes, schemas, interfaces and machine-readable evidence. The model bundle is protected by a recorded SHA-256 value; backup artifacts use authenticated encryption; and administrative activity is recorded for later review.

6.4 Recommendations

- SMEs should begin with at least twelve months of clean product-level sales data where available and review outliers before forecasting.
- Users should maintain realistic supplier lead times, on-hand quantities and reserved stock so order recommendations reflect operations.

- Forecast metrics should be reviewed per product; a single preferred algorithm should not be forced across all series.
- High-risk recommendations should be reviewed promptly, but approval should consider promotions, supplier constraints and business context not present in the model.
- Production operators should keep scheduled health, backup and restore verification workflows active and rotate secrets periodically.

6.5 Future Work

Future work can add promotions, holidays, planned price changes, supplier reliability, weather and economic indicators as model inputs. Hierarchical forecasting could reconcile product, category, store and total-business estimates. Prediction intervals could communicate uncertainty directly, and model drift monitoring could trigger retraining based on measured degradation rather than a fixed schedule.

Operational extensions can include direct integrations with point-of-sale and accounting platforms, supplier purchase-order workflows, offline mobile caching and multilingual guidance. Public mobile distribution can be completed after Apple and Google developer approvals. MongoDB point-in-time recovery can complement the existing encrypted daily backup when the production plan supports it.

6.6 Conclusion

The completed StockGuard AI prototype meets the central commitments of the approved proposal and SRS. It accepts business sales history, compares forecasting models, stores complete future predictions and produces interpretable revenue and replenishment guidance. Its dashboards and mobile application make the analysis accessible, while JWT sessions, permissions, tenant isolation and audit logs protect multi-user business data.

The measured model, application and recovery results show that the design is technically viable for an SME prototype. Most significantly, the system turns a historical CSV file into a traceable purchasing recommendation in one workflow. This connection between prediction, business action and verified software behaviour is the principal outcome of the project.

6.7 Objective-by-Objective Review

Table 20: Achievement of project objectives

Objective	Achievement evidence
Product-level demand forecasting	Saved 7, 14, 28 and 30-day sequences for product-store series
Model comparison	Linear Regression, ARIMA and Random Forest with MAE/RMSE; eligible XGBoost and seasonal candidates
Business decision support	Revenue estimates, safety stock, target stock and recommended order quantity
Secure multi-user access	JWT sessions, roles, permissions, tenant isolation and audit logs
Cross-platform delivery	React web and Expo React Native clients use the same API and workspace
Operational reliability	Health checks, 100-user evidence, encrypted backup and isolated restore

Every primary objective has implementation and verification evidence. The project moved beyond an offline notebook by packaging the trained model, connecting live candidates to the application, persisting forecasts and implementing business actions. The same evidence is visible from both client platforms and is protected by the same authorization boundary.

The strongest engineering contribution is integration. Forecasting quality, inventory policy, secure persistence and usable presentation are often treated separately. StockGuard AI defines a complete

record that connects candidate evaluation, future daily values, revenue, stock context, recommendation and user action. This structure supports later improvement without discarding historical evidence.

6.8 Practical Implications

An SME adopting the system should begin with a limited catalogue and verify that sales dates, quantities, prices and inventory are maintained consistently. Forecasts should be reviewed alongside upcoming promotions and supplier information. As the organization gains confidence, additional stores and users can be added with roles matched to their responsibilities.

The application can reduce time spent assembling weekly spreadsheets and can direct attention toward high-risk products. Its economic benefit will depend on whether users act on the recommendations and whether the configured lead times and safety policy reflect reality. A pilot should therefore track stockout frequency, excess-stock value, forecast error and recommendation outcomes over several ordering cycles.

Administrators should treat health, backup and audit features as routine controls rather than setup tasks. Secrets should be rotated, suspended accounts reviewed and restore evidence checked regularly. Optional SMTP and public mobile distribution can be enabled when the organization selects providers and completes platform approvals.

6.9 Academic Contribution

The dissertation contributes an applied case study of predictive business intelligence for SMEs. It documents how time-series preparation, multiple forecasting candidates and inventory policy can be implemented within a modern multi-tenant application. The work also provides measured evidence rather than limiting evaluation to screenshots or an offline accuracy score.

The project demonstrates the value of requirements traceability in machine-learning software. A model can be accurate while the surrounding application fails to store the full horizon, protect tenant data or recover from loss. By mapping the SRS to routes, schemas, clients and tests, the report evaluates the complete socio-technical system that a business would use.

6.10 Final Conclusion

StockGuard AI demonstrates that an SME-focused platform can convert ordinary historical sales into tested forecasts, transparent replenishment guidance and auditable operational actions. The production prototype satisfies the submitted functional scope, performs within the defined targets and includes the security and recovery controls needed for a credible final-year implementation. Its limitations are explicit, its model evidence is reproducible and its architecture provides a practical foundation for field evaluation and future commercial development.

REFERENCES

[1] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 2nd ed., OTexts, 2018.

[2] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

[3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed., Wiley, 2015.

[4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[6] G. Shmueli and O. R. Koppius, “Predictive Analytics in Information Systems Research,” MIS Quarterly, vol. 35, no. 3, pp. 553–572, 2011.

[7] R. H. Ballou, Business Logistics/Supply Chain Management, 5th ed., Pearson, 2004.

[8] C. C. Holt, “Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages,” International Journal of Forecasting, vol. 20, no. 1, pp. 5–10, 2004 (original memorandum 1957).

[9] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006. (Referenced for systematic software translation and validation concepts.)

[10] OWASP Foundation, OWASP Application Security Verification Standard 4.0.3, 2021.

[11] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.

[12] MongoDB Inc., MongoDB Manual: Data Modeling, Indexes and Security, product documentation, accessed 2026.

[13] Meta Platforms, React and React Native Documentation, product documentation, accessed 2026.

[14] OpenJS Foundation, Node.js Documentation, product documentation, accessed 2026.

[15] I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

[16] NIST, Digital Identity Guidelines: Authentication and Lifecycle Management, SP 800-63B, updated guidance.

CHAPTER A
SRS REQUIREMENT COMPLIANCE

The table below summarises the final implementation status of the eighteen functional requirements submitted in the StockGuard AI SRS. Detailed route and test evidence is retained in docs/SRS_COMPLIANCE.md in the project repository.

Table 21: SRS requirement compliance

Requirement	Description	Status
REQ-1	Registration and validation	Complete
REQ-2	JWT login and dashboard access	Complete
REQ-3	Protected business data	Complete
REQ-4	10 MB / 10,000-row CSV validation	Complete
REQ-5	Product demand forecast	Complete
REQ-6	LR, ARIMA and Random Forest	Complete
REQ-7	MAE/RMSE comparison and selection	Complete
REQ-8	Authorized forecast access	Complete
REQ-9	Product and sales management	Complete
REQ-10	Revenue and replenishment insight	Complete
REQ-11	Responsive analytics	Complete
REQ-12	Upload, forecast and critical alerts	Complete
REQ-13	Administration and activity monitoring	Complete
REQ-14	Audit retention	Complete — one year
REQ-15	Role-based access	Complete
REQ-16	Graceful errors	Complete
REQ-17	Daily backup and recovery	Complete
REQ-18	Privacy and secure communication	Complete

CHAPTER B
VERIFICATION EVIDENCE

The following evidence is reproducible from repository scripts and the deployed application. Test users and verification records are created in isolated tenants and are removed when the check completes.

Verification	Evidence / result
Backend static checks	Route, model, permission and forecasting assertions passed
Authentication integration	Register, login, refresh, logout, revocation and password reset passed
Forecast integration	Thirty daily predictions and candidate metrics saved
Analytics integration	Dashboard aggregations and tenant isolation passed
Web build	Vite production build passed
Mobile check	TypeScript check and Android production export passed
Concurrent production load	100/100 HTTP 200; average 259 ms; p95 832 ms
Recovery	16 isolated collections restored and verified; temporary data removed

CHAPTER C
PRINCIPAL REST API ENDPOINTS

Table 22: Principal REST API endpoints

Method	Path	Purpose
POST	/api/auth/register	Create user and isolated business workspace
POST	/api/auth/login	Authenticate and issue access/refresh session
POST	/api/auth/refresh	Rotate or renew session
GET/PATCH	/api/auth/me	Read or update authenticated profile
GET/POST	/api/products	List or create workspace products
PATCH/DELETE	/api/products/:id	Edit or remove an authorized product
POST	/api/uploads/validate	Validate CSV before commit
POST	/api/uploads	Import a validated sales batch
POST	/api/forecasts/run	Run and save a product/store forecast
GET	/api/forecasts	List saved forecast runs
GET	/api/recommendations	List replenishment actions
PATCH	/api/recommendations/:id/status	Approve, dismiss, complete or reopen
GET	/api/dashboard	Return tenant analytics
GET/POST	/api/reports	List or generate business reports
GET	/api/notifications	List persistent user notifications
GET/PATCH	/api/settings	Read or update business preferences
GET/PATCH	/api/admin/users	Manage users, roles and permissions
GET	/api/admin/audit	Review tenant activity
GET	/api/health	Read production service health

CHAPTER D
INTERNAL DEFENCE DEMONSTRATION SEQUENCE

1. Open the production URL and sign in to the prepared StockGuard AI demonstration workspace.
2. Use the dashboard to explain total products, sales activity, forecast status, stock risks and current recommendations.
3. Open Products and demonstrate product details, inventory quantities, supplier lead time and an edit action.
4. Open CSV Upload, explain the required columns, 10 MB and 10,000-row constraints, and validation feedback.
5. Open AI Forecast, choose a product/store series, select 30 days and run the candidate comparison.
6. Explain MAE, RMSE, the selected model, daily predictions, total demand and forecast revenue.
7. Open the replenishment recommendation and explain lead-time demand, safety stock, target stock and recommended order quantity.
8. Change a recommendation status to demonstrate the approval workflow and audit trail.
9. Open Reports and Insights to show persisted results and business interpretation.
10. Open the mobile application and show that the same tenant data, forecasts and recommendations are available.
11. Finish with Admin, Notifications, Settings and the health/backup evidence to demonstrate operational completeness.

Defence Evidence Summary

Claim	Evidence to show
Forecasting is live	Run a 30-day forecast and inspect all thirty saved daily points
Models are compared	Show LR, ARIMA, RF and eligible production candidates with MAE/RMSE
Inventory advice is calculated	Show safety stock, target stock and order quantity
Web and mobile share data	Open the same product/forecast in both clients
Security is enforced	Show roles, permissions and audit records
System is production-ready	Show health response, load evidence and recovery result

CHAPTER E
SUBMISSION AND DEPLOYMENT CHECKLIST

- Confirm the live web URL opens and the API health endpoint reports MongoDB, Python ML and XGBoost readiness.
- Keep the prepared presentation user credentials private and test login before the defence.
- Open the mobile project through Expo Go on the same network or use a prepared preview build.
- Keep the model bundle, manifest and category mappings in the deployed server package.
- Verify the default forecast horizon is 30 days and that settings are saved for the demonstration workspace.
- Confirm the encrypted backup workflow and production-health workflow show successful recent runs.
- Attach the official similarity report and obtain the required signatures on certificate pages.
- Submit this report together with the approved proposal, SRS, source repository and presentation materials.

CHAPTER F
MONGODB DATA DICTIONARY

This appendix documents the principal persisted entities. Every tenant-owned entity includes a business reference and timestamps unless otherwise stated. Field names are summarized for defence and may be represented by equivalent nested structures in the final Mongoose schema.

F.1 Business Collection

Field	Type	Constraint	Purpose
name	String	Required	Workspace display name
slug	String	Unique	Stable tenant identifier
owner	ObjectId	Required	Owning user reference
status	String	Default active	Workspace lifecycle state
currency	String	Default PKR	Reporting currency
timezone	String	Configured	Date and scheduling context

F.2 User Collection

Field	Type	Constraint	Purpose
name	String	Required	Account display name
email	String	Unique	Normalized login identifier
passwordHash	String	Private	Bcrypt credential hash
phone	String	Optional	Profile contact number
bio	String	Optional	Profile description
sessionVersion	Number	Default 0	Revokes earlier sessions when incremented
status	String	Controlled	Active, invited or suspended state

F.3 Membership Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant reference
user	ObjectId	Required/indexed	User reference
role	String	Required	Owner, admin, manager, analyst or staff
permissions	Array	Validated	Authorized custom permission overrides
status	String	Required	Membership access state
lastActiveAt	Date	Optional	Recent activity marker

F.4 Product Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
sku	String	Unique per business	Stock keeping unit
name	String	Required	Product name
category	String	Optional/indexed	Reporting category
price	Number	Non-negative	Current unit selling price
reorderPoint	Number	Non-negative	Configured threshold
supplierLeadTimeDays	Number	Positive	Expected delivery delay
status	String	Controlled	Active or archived

F.5 Store Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
code	String	Unique per business	External store identifier
name	String	Required	Store display name
location	String	Optional	Human-readable location
status	String	Controlled	Operational state

F.6 Inventory Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
product	ObjectId	Required/indexed	Product reference
store	ObjectId	Required/indexed	Store reference
onHand	Number	Non-negative	Physical recorded quantity
reserved	Number	Non-negative	Committed but not available quantity
updatedBy	ObjectId	Audited	Last modifying user

F.7 Sale Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
product	ObjectId	Required/indexed	Product reference
store	ObjectId	Required/indexed	Store reference
date	Date	Required/indexed	Business sale date
quantity	Number	Non-negative	Units sold
revenue	Number	Non-negative	Recorded sale revenue
upload	ObjectId	Optional	Source import reference

F.8 Upload Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
fileName	String	Required	Original user-facing file name
status	String	Controlled	Validated, completed or failed
rowCount	Number	Non-negative	Parsed rows
insertedCount	Number	Non-negative	Committed rows
errors	Array	Bounded	Row and field validation feedback
uploadedBy	ObjectId	Required	Actor reference

F.9 Forecast Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
product	ObjectId	Required/indexed	Product reference
store	ObjectId	Required/indexed	Store reference
horizonDays	Number	Enum	7, 14, 28 or 30
selectedModel	String	Required	Winning eligible candidate
candidates	Array	Required	Model status, MAE and RMSE
predictions	Array	Exact horizon	Dated daily quantity sequence
totalDemand	Number	Derived	Sum of future quantities
estimatedRevenue	Number	Derived	Demand multiplied by product price
requestedBy	ObjectId	Required	Actor reference

F.10 Recommendation Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
product	ObjectId	Required/indexed	Product reference
store	ObjectId	Required/indexed	Store reference
forecast	ObjectId	Required	Evidence forecast
type	String	Controlled	Low, critical, overstock or spike
status	String	Controlled	Open, approved, dismissed or completed
leadTimeDemand	Number	Derived	Forecast inside supplier lead time
safetyStock	Number	Derived	Variability buffer
targetStock	Number	Derived	Policy inventory target
recommendedOrderQty	Number	Derived	Non-negative order advice

Field	Type	Constraint	Purpose
risk	String	Controlled	Severity for display and alerting

F.11 Report Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
type	String	Required	Sales, inventory, forecast or replenishment
format	String	Required	PDF or CSV
periodStart	Date	Optional	Reporting window start
periodEnd	Date	Optional	Reporting window end
status	String	Controlled	Generation state
generatedBy	ObjectId	Required	Actor reference

F.12 Notification Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant boundary
recipient	ObjectId	Required/indexed	Target user
type	String	Required	Domain notification category
severity	String	Controlled	Information, warning or critical
title	String	Required	Short display title
message	String	Required	Actionable user message
readAt	Date	Optional	Read state timestamp
target	Mixed	Optional	Navigation metadata

F.13 AuditLog Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant context
actor	ObjectId	Optional/indexed	User responsible for the action
action	String	Required/indexed	Normalized event name
resourceType	String	Optional	Affected entity type
resourceId	String	Optional	Affected entity identifier
metadata	Mixed	Sanitized	Non-secret event detail
ip	String	Optional	Limited request context
expiresAt	Date	TTL indexed	One-year retention boundary

F.14 RefreshSession Collection

Field	Type	Constraint	Purpose
user	ObjectId	Required/indexed	Authenticated user
tokenHash	String	Private	Hash of the refresh secret
sessionVersion	Number	Required	Credential lifecycle version
expiresAt	Date	TTL indexed	Session expiry
revokedAt	Date	Optional	Explicit revocation time
device	String	Optional	Limited device description

F.15 Invitation Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Inviting tenant
email	String	Required/indexed	Normalized recipient
role	String	Required	Proposed role
permissions	Array	Validated	Proposed overrides
tokenHash	String	Private	Single-use invitation secret hash
expiresAt	Date	TTL indexed	Invitation expiry
acceptedAt	Date	Optional	Completion timestamp

F.16 BusinessSettings Collection

Field	Type	Constraint	Purpose
business	ObjectId	Unique	Tenant reference
defaultHorizon	Number	Enum	Preferred forecast horizon
defaultModel	String	Controlled	Preferred eligible candidate
confidenceThreshold	Number	Bounded	Display and policy threshold
safetyStockPercent	Number	Bounded	Replenishment buffer setting
notificationPreferences	Mixed	Validated	Channel and category controls
appearance	String	Controlled	Theme preference

F.17 PushDevice Collection

Field	Type	Constraint	Purpose
business	ObjectId	Required/indexed	Tenant context
user	ObjectId	Required/indexed	Device owner
expoPushToken	String	Unique	Expo delivery token
platform	String	Controlled	iOS or Android
enabled	Boolean	Default true	Delivery status
lastSeenAt	Date	Updated	Token activity marker

CHAPTER G
DETAILED VERIFICATION CATALOGUE

The catalogue records representative positive, negative, authorization and recovery tests used to assess the delivered system. Status describes the verified result at the final project checkpoint.

G.1 Test Cases 1 to 14

ID	Scenario	Expected / observed	Status
AUTH-01	Register with valid fields	201 and isolated workspace	Pass
AUTH-02	Register duplicate email	Conflict without duplicate user	Pass
AUTH-03	Register malformed email	Validation error	Pass
AUTH-04	Login valid credentials	Access and refresh session	Pass
AUTH-05	Login invalid password	Generic unauthorized response	Pass
AUTH-06	Refresh valid session	Rotated access session	Pass
AUTH-07	Reuse revoked refresh	Unauthorized	Pass
AUTH-08	Logout	Refresh session revoked	Pass
AUTH-09	Password change	Old sessions invalidated	Pass
AUTH-10	Password reset expiry	Expired token rejected	Pass
TEN-01	Read another tenant product	Not found or forbidden	Pass
TEN-02	Update another tenant store	No modification	Pass
TEN-03	List dashboard records	Current business only	Pass
RBAC-01	Staff opens admin users	Forbidden	Pass

G.2 Test Cases 15 to 28

ID	Scenario	Expected / observed	Status
RBAC-02	Analyst runs forecast	Allowed with permission	Pass
RBAC-03	Custom permission removed	Action becomes forbidden	Pass
PROD-01	Create valid product	Tenant product stored	Pass
PROD-02	Duplicate SKU	Conflict per business	Pass
PROD-03	Negative product price	Validation error	Pass
PROD-04	Edit product	Updated values and audit event	Pass
PROD-05	Delete/archive product	Controlled lifecycle result	Pass
UPL-01	Validate supported CSV	Preview and counts returned	Pass
UPL-02	File above 10 MB	Rejected before import	Pass
UPL-03	More than 10,000 rows	Limit error	Pass
UPL-04	Missing required header	Header error	Pass
UPL-05	Invalid date	Row-specific error	Pass
UPL-06	Invalid quantity	Row-specific error	Pass
UPL-07	Duplicate sales key	Duplicate prevented	Pass

G.3 Test Cases 29 to 42

ID	Scenario	Expected / observed	Status
UPL-08	Insertion failure	Batch rollback	Pass
FC-01	Seven-day forecast	Seven dated predictions	Pass
FC-02	Fourteen-day forecast	Fourteen dated predictions	Pass
FC-03	Twenty-eight-day forecast	Twenty-eight dated predictions	Pass
FC-04	Thirty-day forecast	Thirty dated predictions	Pass
FC-05	Linear Regression candidate	MAE and RMSE returned	Pass
FC-06	ARIMA candidate	Metric or controlled unavailable state	Pass
FC-07	Random Forest candidate	MAE and RMSE returned	Pass
FC-08	Mapped XGBoost series	Verified booster candidate	Pass
FC-09	Unmapped identifier	Fallback without invalid encoding	Pass
FC-10	One candidate failure	Other candidates continue	Pass
FC-11	Metric selection	Lowest MAE selected	Pass
FC-12	Revenue estimate	Demand multiplied by price	Pass
REC-01	Lead-time demand	Correct forecast subset summed	Pass

G.4 Test Cases 43 to 56

ID	Scenario	Expected / observed	Status
REC-02	Safety stock	Variability and lead time applied	Pass
REC-03	Target stock	Coverage and buffer combined	Pass
REC-04	Recommended quantity	Non-negative stock gap	Pass
REC-05	Duplicate open action	Existing record updated	Pass
REC-06	Approve action	Status and actor recorded	Pass
REC-07	Dismiss action	History retained	Pass
REC-08	Complete action	Lifecycle completed	Pass
NOT-01	Upload completed	Persistent notification created	Pass
NOT-02	Forecast ready	Persistent notification created	Pass
NOT-03	Critical stock	Critical alert created	Pass
NOT-04	Email not configured	Primary transaction still succeeds	Pass
ADM-01	Invite user	Tokenized invitation and audit event	Pass
ADM-02	Edit role	Permissions update immediately	Pass
ADM-03	Suspend member	Future access blocked	Pass

G.5 Test Cases 57 to 70

ID	Scenario	Expected / observed	Status
ADM-04	Audit list	Tenant events with filters	Pass
REP-01	Generate forecast report	Saved downloadable report	Pass
REP-02	Unauthorized report	Access denied	Pass
WEB-01	Production build	Vite build completes	Pass
WEB-02	Session restore	Protected page opens after refresh	Pass
MOB-01	TypeScript validation	No type errors	Pass
MOB-02	Android export	Production bundle generated	Pass
PERF-01	100 authenticated users	No failed response	Pass
PERF-02	p95 response	Below three seconds	Pass
PERF-03	Forecast response	Below ten seconds	Pass
BKP-01	Encrypt snapshot	Authenticated artifact created	Pass
BKP-02	Incorrect key	Decryption rejected	Pass
BKP-03	Modified artifact	Authentication check fails	Pass
BKP-04	Isolated restore	Collections and counts verified	Pass

G.6 Test Cases 71 to 71

ID	Scenario	Expected / observed	Status
BKP-05	Cleanup	Temporary collections removed	Pass

CHAPTER H
API RESOURCE CATALOGUE

All protected endpoints require an authenticated session and enforce tenant ownership and permissions appropriate to the action. The catalogue summarizes the final public business API used by the web and mobile clients.

H.1 Endpoint Group 1

Method	Path	Purpose
POST	/api/auth/register	Create user, business and owner membership
POST	/api/auth/login	Authenticate and issue session
POST	/api/auth/refresh	Rotate access using valid refresh session
POST	/api/auth/logout	Revoke current refresh session
POST	/api/auth/forgot-password	Create time-limited reset request
POST	/api/auth/reset-password	Set new password and revoke older sessions
GET	/api/auth/me	Return authenticated profile and membership
PATCH	/api/auth/me	Update allowed profile fields
PATCH	/api/auth/password	Verify current password and rotate credentials
GET	/api/products	List and filter tenant products
POST	/api/products	Create a product
GET	/api/products/:id	Read product and inventory context
PATCH	/api/products/:id	Update product configuration
DELETE	/api/products/:id	Remove or archive product under policy
GET	/api/stores	List tenant stores

H.2 Endpoint Group 2

Method	Path	Purpose
POST	/api/stores	Create store
PATCH	/api/inventory/:id	Update on-hand and reserved quantities
GET	/api/sales	List authorized sales
POST	/api/sales	Create validated sale
POST	/api/uploads/validate	Validate CSV without committing
POST	/api/uploads	Import validated sales records
GET	/api/uploads	List upload history
POST	/api/forecasts/run	Compare candidates and save forecast
GET	/api/forecasts	List saved forecasts
GET	/api/forecasts/:id	Read candidate and daily forecast detail
GET	/api/recommendations	List and filter replenishment actions
PATCH	/api/recommendations/:id/status	Apply authorized lifecycle transition
GET	/api/dashboard	Return aggregated business overview
GET	/api/insights	Return evidence-based business insights
GET	/api/reports	List saved reports

H.3 Endpoint Group 3

Method	Path	Purpose
POST	/api/reports	Generate report
GET	/api/reports/:id/download	Download authorized report content
GET	/api/notifications	List recipient notifications
PATCH	/api/notifications/:id/read	Mark notification read
POST	/api/notifications/push-device	Register Expo push token
GET	/api/settings	Read business and user preferences
PATCH	/api/settings	Update allowed preferences
GET	/api/admin/users	List tenant memberships
POST	/api/admin/invitations	Invite a user
PATCH	/api/admin/users/:id	Edit role, permission or status
GET	/api/admin/audit	Review audit events
GET	/api/admin/activity	Review system activity summary
GET	/api/health	Return service readiness

H.4 Representative Forecast Request

A forecast request supplies the tenant-owned product and store identifiers together with a supported horizon. The server ignores any attempt to substitute a business identifier in the body and derives tenancy from the session. The successful representation includes the selected model, candidate status and metrics, thirty dated values for a 30-day request, total demand, estimated revenue and linked replenishment evidence.

H.5 Error Contract

Validation errors use a stable JSON structure containing an error code, human-readable message and optional field details. Authentication and authorization failures do not reveal password, membership or cross-tenant resource information. Dependency errors identify temporary service unavailability without returning internal stack traces. Unexpected exceptions are logged with a correlation context and converted to a controlled server response.

CHAPTER I
USER OPERATIONS GUIDE

This appendix provides a concise operational sequence for the deployed prototype. Exact labels may adapt to screen width, but the underlying workflow is shared by web and mobile clients.

I.1 Sign In

1. Open the production application.
2. Enter the registered email and password.
3. Submit the form and wait for the protected dashboard.
4. If credentials are rejected, verify the account status or start password recovery.
5. Use Logout when leaving a shared device.

I.2 Review the Dashboard

1. Confirm the active business workspace.
2. Review product, sales, forecast and revenue summary cards.
3. Check low-stock and critical indicators.
4. Open a recommendation or notification requiring attention.
5. Change the date window where the page supports it.

I.3 Maintain Products

1. Open Products and search by name or SKU.
2. Select Add Product and enter category, price, reorder point and lead time.
3. Save and confirm the product appears in the tenant list.
4. Use Edit to correct controlled fields.
5. Use delete/archive only after considering linked history.

I.4 Update Inventory

1. Open the selected product or inventory action.
2. Choose the correct store.
3. Enter on-hand and reserved units.
4. Save and verify available stock.
5. Review whether the updated quantity changes current risk.

I.5 Validate and Upload Sales

1. Prepare CSV fields according to the example format.
2. Keep the file below 10 MB and 10,000 rows.
3. Select the file and run validation.
4. Correct every blocking row error before import.
5. Commit the valid batch and confirm upload history and notification.

I.6 Run a Forecast

1. Open AI Forecast.
2. Select a product and store with sufficient history.
3. Choose 7, 14, 28 or 30 days.
4. Run the comparison and wait for the saved result.
5. Review the selected model, MAE, RMSE, daily chart, total demand and revenue.

I.7 Review Replenishment

1. Open the recommendation linked to the forecast.
2. Check current and reserved stock, lead time and variability.
3. Review safety stock, target stock and recommended order quantity.
4. Approve, dismiss or later complete the action according to business context.
5. Use the audit or activity view when confirmation is required.

I.8 Generate a Report

1. Open Reports and choose sales, inventory, forecast or replenishment.
2. Select a valid period and format.
3. Generate the report and wait for completion.
4. Download the saved report from the authorized list.
5. Confirm that figures match the selected business and period.

I.9 Manage Notifications

1. Open Notifications from the header or mobile menu.
2. Read critical items first.
3. Open the linked target when available.
4. Mark items read after review.
5. Use Settings to enable or disable supported categories and channels.

I.10 Manage Users

1. Open Admin with an owner or administrator account.
2. Invite a user with the minimum suitable role.
3. Adjust explicit permissions only when the role default is insufficient.
4. Suspend access when a member should no longer sign in.
5. Review the audit log after sensitive changes.

I.11 Edit Profile and Security

1. Open Profile to update allowed personal details.
2. Use Security settings to change the password.
3. Expect existing sessions to be revoked after a password change.
4. Review new-login notifications.
5. Do not share refresh credentials or API secrets.

I.12 Mobile Access

1. Install Expo Go for development access or use the prepared build.
2. Open the project link while network access is available.
3. Sign in with the same StockGuard account.
4. Review forecasts and recommendations from the shared workspace.
5. Allow notifications only when mobile alerts are desired.

I.13 Mobile Troubleshooting

If Expo Go displays an opening-project spinner for an extended period, confirm that the computer and phone can reach the selected Metro address. LAN mode requires a reachable local network, while tunnel mode requires a successful Expo account session and tunnel service. The user should not repeatedly create new projects or modify application code to resolve a network-only problem. Restarting Metro with a cleared cache and rescanning the current QR code is the controlled first step.

The same React Native code supports Android and iOS through Expo. Local iOS simulation requires macOS, but a physical iPhone can use Expo Go during development and an EAS iOS build when Apple credentials are available. API calls always use the configured HTTPS production address for the presentation build; a private computer localhost address is not reachable from the phone. Authentication errors should be separated from connectivity errors by checking the API health endpoint and then the account credentials.

CHAPTER J
SECURITY THREAT MODEL

The threat model applies practical controls to the deployed academic prototype. Residual risk identifies conditions that require continued operations or user discipline.

J.1 Threat Register 1

Threat	Impact	Control	Residual risk
Credential guessing	Account compromise	Bcrypt, generic login errors and monitored events	Rate limiting and user password quality remain important
Stolen access token	Temporary unauthorized requests	Short expiry and HTTPS	Valid until expiry unless surrounding session is revoked
Stolen refresh token	Repeated token creation	HttpOnly/SecureStore, hash storage and rotation	Compromised device requires session revocation
Cross-tenant identifier	Business data disclosure	Server-derived tenant and ownership queries	New routes must preserve the pattern
Privilege escalation	Unauthorized administration	Permission middleware and membership status	Misconfigured custom permission must be reviewed
CSV formula or malformed data	Corruption or unsafe export	Strict parsing, type validation and controlled output	Source spreadsheet still requires safe handling
Duplicate import	Inflated demand	Duplicate keys and idempotency logic	Changed source identifiers can bypass semantic duplicate detection
Injection payload	Database or UI compromise	Schema validation and React escaping	Dependencies and new rendering paths require review
Model artifact replacement	Manipulated predictions	Version manifest and SHA-256 check	Host write access must remain restricted
Python service spoofing	False candidate output	Private configured endpoint and response validation	A broader service mesh is outside prototype scope

J.2 Threat Register 2

Threat	Impact	Control	Residual risk
Sensitive log content	Credential disclosure	Sanitized metadata and no password/token logging	Developer-added logs require review
Backup theft	Historical data disclosure	AES-256-GCM encrypted artifacts	Encryption key protection is critical
Backup tampering	Invalid recovery	Authenticated encryption and restore checks	Availability still depends on retained artifact
Destructive administrator	Tenant data loss	Permissions, audit trail and backup	Authorized misuse cannot be eliminated completely
Denial of service	Reduced availability	Upload limits, bounded requests and cloud concurrency	Large-scale hostile traffic needs platform controls
Dependency vulnerability	Application compromise	Lockfiles, audits and update workflow	New advisories require repeated checks
Push-token leakage	Unwanted notification delivery	Tenant/user association and disable state	External push provider processes token
Email misdelivery	Information sent to wrong address	Verified account email and preferences	SMTP provider and mailbox security are external
Lost mobile device	Session exposure	SecureStore and revocable sessions	Device-level lock remains the user's responsibility
Recovery error	Live data overwritten	Prefixed isolated restore	Manual emergency restoration requires controlled procedure

CHAPTER K
REQUIREMENTS TRACEABILITY MATRIX

Requirement	Scope	Implementation	Verification
REQ-1	Registration	Auth routes, User/Business/Membership	AUTH-01 to AUTH-03
REQ-2	JWT login	Login, refresh and session models	AUTH-04 to AUTH-08
REQ-3	Protected business data	Hashing, HTTPS and tenant middleware	AUTH and TEN suites
REQ-4	CSV limits and validation	Upload parser and validation service	UPL-01 to UPL-08
REQ-5	Product demand forecast	Forecast route and persistence	FC-01 to FC-04
REQ-6	LR, ARIMA and RF	Python candidate service	FC-05 to FC-07
REQ-7	MAE/RMSE selection	Candidate selector	FC-10 and FC-11
REQ-8	Authorized forecasts	Forecast permissions and tenant checks	RBAC-02 and TEN suite
REQ-9	Product management	Product and inventory routes	PROD-01 to PROD-05
REQ-10	Revenue and replenishment	Revenue/recommendation services	FC-12 and REC suite
REQ-11	Responsive analytics	Dashboard API and React views	WEB-01 and analytics checks
REQ-12	Operational alerts	Notification service and preferences	NOT-01 to NOT-04
REQ-13	Administration	Invitation/user/activity routes	ADM-01 to ADM-04
REQ-14	Audit retention	AuditLog TTL and list route	ADM-04
REQ-15	Role-based access	Membership and permission middleware	RBAC-01 to RBAC-03
REQ-16	Graceful errors	Validation and central error middleware	Negative cases across suites
REQ-17	Backup and recovery	Encrypted workflow and restore script	BKP-01 to BKP-05
REQ-18	Privacy and communication	Tenant isolation, JWT and HTTPS	TEN and security suites

K.1 Traceability Interpretation

The matrix demonstrates bidirectional traceability: each submitted requirement has an implementation location and verification evidence, while the major implemented services map back to an approved requirement. This structure supports internal defence questions and reduces the chance that a polished interface is mistaken for completion without underlying behaviour.

CHAPTER L
OPERATIONAL READINESS CHECKLIST

- Production web URL opens over HTTPS and protected routing restores a valid session.
- API health reports database connectivity and current forecasting readiness.
- MongoDB URI belongs to the StockGuard-specific user and minimum required database role.
- Access and refresh secrets are configured outside source control and can be rotated.
- The XGBoost model, manifest and mappings are present and the model hash is verified.
- Python forecasting service URL is configured and candidate timeout handling is active.
- Presentation tenant contains products, sales, forecast, reports, notifications and recommendations.
- The default horizon is set to thirty days for the required demonstration.
- Web production build and mobile TypeScript/export checks pass from a clean checkout.
- The latest one-hundred-user performance evidence remains attached to the submission.
- The latest encrypted backup completed and isolated restore evidence is available.
- Temporary test tenants and restore collections are removed after verification.
- SMTP is configured only if real email delivery will be demonstrated.
- Expo push credentials and a registered device are available if physical push receipt will be demonstrated.
- The official similarity report and committee signature pages are attached before final university submission.
- A local copy of the report, source package, model bundle and presentation is retained separately.

L.1 Defence Demonstration Timing

Time	Demonstration activity	Evidence
0-2 min	Problem, users and architecture	Cover, dashboard and architecture diagram
2-5 min	Products and validated sales upload	Catalogue and validation feedback
5-9 min	Thirty-day model comparison	Candidate metrics and thirty daily values
9-11 min	Replenishment decision	Safety stock, target and order quantity
11-13 min	Reports, notifications and mobile	Shared tenant data on both clients
13-15 min	Security, performance and recovery	Roles, audit, load and restore evidence

CHAPTER M
ALGORITHM SPECIFICATIONS

This appendix describes the principal algorithms at a level suitable for implementation review. It records inputs, processing decisions, outputs and edge-case behaviour so that the reported workflow can be understood independently of a particular user interface.

M.1 Daily-Series Construction

The daily-series constructor accepts an authenticated business identifier, product identifier, store identifier and chronological range. It queries only sales that match all ownership fields. Individual transactions occurring on the same date are aggregated into one quantity. The date index is then expanded from the first through the last observed day so every calendar day has a position in the sequence.

A missing calendar day inside the observed range is represented as zero demand because no valid transaction was recorded for the series on that date. This rule is applied only after CSV validation has distinguished a missing record from an invalid date. The constructor returns ordered date and quantity arrays together with descriptive values including observation count, non-zero count, mean, standard deviation and last observation date.

The algorithm rejects a series whose product or store does not belong to the active business. It also rejects an empty history and reports insufficient data when the valid candidate set cannot be evaluated. These checks occur before external model calls. The result is a deterministic input shared by all candidates, ensuring that model comparison is not affected by different preprocessing paths.

Table 23: Daily-series construction specification

Element	Specification
Input	Business, product, store, date range and validated Sale documents
Ordering	Ascending calendar date
Aggregation	Sum quantities for identical tenant/product/store/date keys
Missing dates	Insert zero quantity inside the observed date interval
Output	Dates, quantities and descriptive history metadata
Failure	Ownership mismatch, empty history or insufficient candidate data

M.2 Holdout Construction

The holdout constructor reserves the most recent observations because production use always predicts dates after the available history. Its size is bounded by the series length and the evaluation policy. Enough earlier rows must remain to compute the largest lag and to train each candidate. When a candidate needs more history than is available, it returns an unavailable status rather than reducing the common holdout silently.

All eligible candidates receive the same training-end date and actual holdout quantities. Metrics are computed only after predicted dates are aligned with actual dates. This prevents a shorter successful candidate from being rewarded for predicting fewer or easier observations. Candidate output includes status, reason where unavailable, MAE, RMSE and prediction count.

Time order is never shuffled. Scaling or feature fitting, when required, is fitted on the training portion and applied to holdout or future rows. The project does not use future actual quantities to

construct lag or rolling inputs. During recursive holdout evaluation, earlier candidate predictions are appended in the same way that future production predictions will be appended.

M.3 Linear Regression Candidate

The Linear Regression candidate transforms each eligible date into a row containing time index, lagged demand, rolling means and cyclical calendar variables. Rows with unavailable initial lags are excluded. The estimator minimizes squared residuals on the training rows. Although the model is simple, lag and calendar features allow it to represent more than a straight line through quantity over time.

Holdout prediction is recursive. For each date, features are built from the training history and any earlier predicted holdout values. The result is clipped at zero and appended before the next date is processed. After the holdout metrics are measured, the candidate fits on all available supervised rows and repeats the recursive process for the requested future horizon.

The candidate is useful as a fast and interpretable baseline. Coefficients can indicate whether recent demand, weekly lag or trend contributes to the result, but collinearity among lag and rolling variables can make an individual coefficient unstable. The project therefore uses predictive metrics for selection rather than treating coefficient magnitude as a causal conclusion.

M.4 ARIMA Candidate

ARIMA models a univariate ordered series through autoregressive terms, differencing and moving-average error terms. Differencing is intended to remove a persistent level change so that the remaining structure is more stable. Autoregressive terms relate the current value to previous values, while moving-average terms relate it to earlier residuals. The candidate uses a conservative specification appropriate to the available history and catches fitting or convergence failures.

The model is fitted only on the training sequence for holdout evaluation and generates the required number of steps. Predictions are aligned to the holdout dates, clipped to the non-negative domain and evaluated with MAE and RMSE. For production, the selected ARIMA specification is refitted on the complete observed sequence and asked for the requested horizon.

ARIMA can represent autocorrelation without the engineered tree inputs used by Random Forest and XGBoost. It may perform well for a stable series with systematic temporal dependence. It may be unavailable or weak for a very short, all-zero or highly irregular series. Such a result is retained as candidate evidence and does not prevent other models from completing the request.

M.5 Random Forest Candidate

Random Forest trains multiple regression trees on bootstrap samples and averages their predictions. At each split, a subset of features is considered, reducing correlation among trees. The model can learn nonlinear rules such as a demand increase that occurs only when a particular lag and calendar condition appear together. It also requires little distributional assumption.

The StockGuard candidate uses the same leakage-safe lag, rolling and cyclical feature rows as Linear Regression. Random state and core hyperparameters are fixed for reproducibility in the academic prototype. Holdout and future sequences are generated recursively. The forest's inability to extrapolate far beyond values represented in its leaves can be beneficial for stability but may understate a new upward trend.

Feature importance can be inspected as a diagnostic, but impurity-based importance is not treated as causal evidence and can favour continuous or high-variance inputs. Selection remains based on holdout MAE and RMSE. The candidate returns model metadata sufficient for the UI to explain that the result came from the locally fitted Random Forest path.

M.6 Global XGBoost Candidate

The XGBoost bundle is a gradient-boosted tree model trained across product-store series. Each new tree focuses on residual patterns left by the existing ensemble. Regularization, learning rate and early stopping control complexity. The final bundle contains 83 boosting rounds and expects twelve features in the recorded order: store code, product code, four lags, two rolling values and four cyclical calendar values.

Before inference, the runtime loads the manifest, category mappings and model JSON. It computes SHA-256 and compares it with the manifest value. A mismatch disables the candidate because the relationship between code, mapping and trained trees can no longer be trusted. The runtime also verifies the expected XGBoost version and feature list for diagnostics.

Store and product identifiers are mapped with the training dictionaries. Unknown identifiers make the candidate ineligible rather than being encoded as an existing category. For an eligible series, the runtime builds a numeric row in the exact feature order and executes CPU inference. Recursive future values are appended to history for later lag and rolling construction.

The global model's accuracy evidence comes from three chronological recursive folds. Production training then uses data through 31 October 2019 and produces the packaged 28-day reference forecast from 1 to 28 November 2019. The application extends supported request horizons through recursive inference and verifies complete 30-day persistence at the API level.

M.7 Seasonal-Naive Candidate

Seasonal naive predicts a future value from the corresponding observation in the previous seasonal cycle, with a weekly period used for daily retail demand where sufficient history exists. If the exact seasonal lag is not available, a guarded recent value or average provides a conservative continuation. The algorithm has almost no fitted parameters and is therefore resilient for a small or irregular series.

The candidate is both a benchmark and a fallback. Beating it demonstrates that a more complex model adds value beyond repeating recent weekly behaviour. Retaining it in production means a forecast can still be generated when a statistical model fails to converge, the Python service is temporarily unavailable or an identifier is not eligible for global XGBoost.

Fallback status is stored explicitly. The interface does not label a seasonal result as XGBoost or Random Forest merely to preserve a preferred name. Transparent model identity is necessary for evaluation and for later comparison of recommendation outcomes.

M.8 Candidate Selection

1. Collect every candidate response and normalize success, unavailable and failure states.
2. Remove candidates that do not contain a complete metric pair and aligned holdout predictions.
3. Sort successful candidates by ascending MAE.
4. When MAE is equal within numeric tolerance, sort by ascending RMSE.

5. Use a deterministic candidate order only when both metrics remain equal.
6. Generate or accept the complete future sequence from the winner.
7. Verify that the number of unique consecutive future dates equals the requested horizon.
8. Persist every candidate state, the winner, predictions and summary totals in one forecast record.

The selector does not convert MAE into an unsupported percentage accuracy. Where the interface provides an explanatory confidence label, the underlying MAE, RMSE, sample and model identity remain available. This protects the user from interpreting a visually convenient score as a statistical guarantee.

M.9 Replenishment Algorithm

The replenishment input combines the selected forecast, current inventory, reserved quantity, supplier lead time, configured safety percentage and historical demand. Available inventory is max(0, on-hand minus reserved). Lead-time demand is the sum of predictions during the lead-time window. If lead time extends past the saved horizon, the remaining portion is estimated conservatively from the available future daily rate and recorded as an assumption.

Daily variability is calculated from historical quantities after the same calendar treatment used for forecasting. Safety stock increases with variability and the square root of lead time. The review period represents the interval until the next planned order review. Target stock covers lead time, review-period demand and safety stock. Order quantity is the rounded non-negative difference between target and available inventory.

The algorithm returns components separately, allowing a manager to reproduce the conclusion. Risk classification considers days of cover, reorder point, immediate demand and spike ratio. The recommendation service finds an existing open record with the same tenant, product, store and type. It refreshes evidence or creates a new record, then emits a high-risk notification when the classification requires it.

M.10 Report Generation Algorithm

A report request specifies an allowed report type, format and optional date window. The service verifies report permissions and business ownership before querying data. Sales reports group quantities and revenue by time and category. Inventory reports include available stock, reorder position and open recommendation. Forecast reports include model, metrics, daily values and totals. Replenishment reports include the formula components and action status.

The generated file is linked to a Report document containing type, period, format, actor, status and storage metadata. Download routes repeat authorization rather than relying on possession of a client-side URL. Numbers are formatted for readability while the source values remain stored in their original numeric form. A generation failure updates status and returns a controlled error without creating a misleading completed report.

CHAPTER N
DETAILED SYSTEM DESIGN

N.1 Architectural Responsibilities

The presentation layer owns rendering, navigation, local form state and accessible feedback. It does not decide tenant identity, calculate authoritative replenishment quantities or choose a model. Those responsibilities remain on the server so web and mobile behaviour cannot diverge and a modified client cannot bypass policy.

The API layer owns authentication, authorization, validation, orchestration and persistence. Route handlers translate HTTP input into service calls, middleware establishes security context and services implement domain rules. Database models express structural constraints and indexes. External delivery and model clients are adapters whose failures are converted into controlled domain outcomes.

The Python forecasting layer owns candidate implementations that depend on Pandas, scikit-learn and Statsmodels. It accepts normalized series input and returns candidate results; it does not access MongoDB or issue business sessions. The XGBoost runtime is packaged with the Node deployment so the verified booster can be used without transferring the model bundle on every request.

MongoDB Atlas provides persistent tenant data. Vercel hosts the browser application and serverless API surfaces, while Expo supports mobile development and builds. GitHub stores versioned source and executes scheduled operational workflows. These boundaries limit the credentials and responsibilities required by each component.

N.2 Request Lifecycle

An HTTPS request enters the API through platform routing. Security middleware attaches a request identifier, parses the supported credential source and verifies the access token. Membership status and session version are loaded or validated from claims according to route policy. Permission middleware then checks the requested action.

Validation parses route parameters, query filters and body content into an expected schema. Unknown or malformed fields are rejected before service execution. The service builds database queries with the authenticated business identifier. A resource-specific route confirms ownership even when the identifier format is valid.

Domain work may call a model or notification adapter, but the service remains responsible for persistence order. The response serializer removes private fields and produces a stable JSON representation. Errors pass to central middleware, which selects an appropriate status and public message while preserving diagnostic context in controlled logs.

N.3 Authentication Sequence

During login, the server normalizes the email and loads the credential record. Password comparison uses bcrypt and a uniform unauthorized response. Active membership and business status are checked before a session is issued. The access token is short-lived; the refresh secret is represented by a server-side hash and expiry. Web and mobile clients receive it through storage appropriate to their platform.

When the access token expires, the API client performs one refresh operation and queues concurrent failed requests rather than issuing multiple rotations. A valid refresh session is checked for expiry, revocation and session version. Rotation invalidates the previous refresh value, reducing the replay window. The original request is retried with the new access token.

Password reset uses a random, single-use secret sent through the selected delivery process. Only its hash is stored. Successful reset replaces the password hash, consumes the reset record and increments session version. Password change requires the current password and follows the same session invalidation principle. Logout revokes the current refresh session even if the access token remains valid for its short remaining lifetime.

N.4 Forecast Sequence

The forecast screen loads authorized products, stores and recent saved runs. When the user starts a run, the server verifies forecast permission and loads the tenant-owned series. Series construction and sufficiency checks complete before the Python request. Eligible candidate tasks can execute independently, but selection waits for all bounded results.

The Python response is validated for model names, finite metrics and aligned prediction arrays. The XGBoost and seasonal candidates use the same holdout policy in Node. Failed candidates retain a safe reason suitable for diagnostics without exposing a stack trace. The selector chooses the winner and validates the future sequence.

A transaction-like service stores the forecast and applies replenishment logic. Recommendation and notification operations use the saved forecast identifier. The API returns the complete stored representation. The client updates its list and detail views from the response, avoiding a second calculation of totals in presentation code.

N.5 Recommendation Sequence

Recommendation generation begins after a forecast or an authorized recalculation. The service loads current inventory and business settings at the time of calculation. Formula components and risk rules produce an evidence object. A lookup for an open matching action is performed under the tenant key.

When a match exists, the evidence, risk and forecast link are refreshed while user workflow history remains intact. Otherwise, a new open action is created. A critical action creates persistent notifications for the intended recipients and attempts configured email or push delivery after the primary record exists.

Status changes verify transition permission and the current state. Approved marks managerial acceptance, dismissed records a decision not to act and completed indicates operational follow-through. Reopen is available to an authorized role when policy permits. Every transition creates an audit event with actor and target.

N.6 Reporting Sequence

The user chooses report type, time window and format. The server validates the period and authorizes access. Aggregation pipelines calculate the relevant totals using tenant and date filters. The report renderer formats headings, data rows and metadata and stores the generated artifact or content reference.

The report list shows only records belonging to the current business and permitted type. Download repeats authorization and streams the expected content type. Historical saved reports remain stable even when a later forecast changes, supporting audit and comparison. A report can be regenerated for an updated period rather than silently changing the earlier artifact.

N.7 Notification Sequence

A domain event creates one or more recipient-specific Notification documents. Preference resolution checks category and channel settings. In-app delivery is complete when the persistent record exists. External adapters receive a minimal payload containing title, message and supported navigation metadata.

Email and push outcomes are logged separately. Invalid mobile tokens can be disabled after a permanent provider response. Reading an in-app item sets readAt for the authenticated recipient only. Bulk read actions remain tenant- and user-scoped. The header count and mobile badge are calculated from unread recipient records.

N.8 Backup and Restore Sequence

The scheduled workflow obtains the StockGuard-specific MongoDB connection from encrypted repository secrets and exports the selected database. It records metadata required to interpret the snapshot, then encrypts the archive using AES-256-GCM with a fresh nonce. The authentication tag is stored with the encrypted artifact. Plain temporary output is removed after encryption.

Verification decrypts a controlled copy, rejects a wrong tag and imports documents with a unique temporary prefix. The script enumerates collections, confirms document counts and performs readability checks. It never restores to the production collection names during the routine drill. After evidence is recorded, every prefixed collection and temporary file is removed.

An actual disaster recovery would use the verified artifact and key in a controlled environment, confirm business ownership and counts, and then perform a separately approved cutover. The routine drill proves archive usability while avoiding the risk of accidental replacement during normal operation.

N.9 Deployment Topology

The browser downloads static assets from the web deployment and calls same-project API routes over HTTPS. The API connects to MongoDB Atlas using a restricted database account and invokes the configured Python forecasting endpoint when a live candidate comparison is requested. Static XGBoost artifacts are deployed with the runtime package.

The React Native application calls the public HTTPS API directly and registers Expo push tokens. It does not connect to MongoDB. GitHub Actions connects only during scheduled monitoring or backup with separate secrets. This topology prevents database credentials from reaching either client and keeps the Node API as the policy enforcement point.

N.10 Design Trade-offs

Table 24: Major architectural trade-offs

Decision	Benefit	Cost / constraint	Reason accepted
Separate Python service	Mature statistics libraries	Extra deployment dependency	Required live LR, ARIMA and RF support

Decision	Benefit	Cost / constraint	Reason accepted
Node policy boundary	One security and persistence layer	Orchestration complexity	Consistent web/mobile behaviour
MongoDB documents	Natural nested forecasts and candidates	Relational joins require deliberate aggregation	Fits bounded analytical records
Recursive multi-step forecast	Supports arbitrary required horizons	Prediction error can accumulate	Matches available one-step feature models
Stored daily predictions	Traceability and lead-time sums	More document data	Required for charts and verification
Explicit seasonal fallback	Resilience and benchmark	Simpler pattern	Prevents complete model-path failure
Advisory recommendations	Human control	Requires user follow-through	Appropriate for incomplete business context
Daily encrypted backup	Portable and verifiable	Up to daily recovery-point interval	Meets submitted prototype requirement

CHAPTER O
USER INTERFACE SPECIFICATION

O.1 Sign-In Screen

The sign-in screen presents the StockGuard logo, project identity, email field, password field, submit action and recovery link. Fields use explicit labels and browser/mobile input types. The submit action is disabled or guarded while a request is active. Invalid credentials produce a generic message, while network failure provides a retry path.

A valid session redirects to the dashboard. If a protected URL was requested before authentication, the application may preserve a safe internal destination. Session restoration displays an intentional loading state so the user does not see an empty dark screen. Google sign-in remains outside the implemented scope unless a provider is configured; email and password are the verified path.

O.2 Dashboard

The dashboard summarizes total products, sales, forecast activity, revenue and inventory risk. Cards show a value with a clear label and change context where available. Analytical charts use tenant data from the selected window. A recommendation area prioritizes critical and low-stock actions, and recent notifications provide an operational entry point.

Desktop layout uses a multi-column grid, while narrow viewports stack cards and charts. Loading skeletons preserve page structure. Empty states explain which setup action is required, such as adding products or uploading sales, instead of presenting a zero without context. Errors offer a reload action and do not discard the authenticated session unnecessarily.

O.3 Products

The product page supports search, category filtering, health status and authorized create, edit or delete/archive actions. Each record exposes SKU, category, price, stock, reorder point, supplier and risk. Status combines text and colour. Actions are hidden or disabled when the user lacks permission, although the server remains authoritative.

The product form validates SKU, name, category, price, reorder point and lead time. Existing identifiers are populated for editing. Confirmation feedback identifies the saved product. A detail view connects inventory, recent sales, forecasts and recommendations so the user can understand the source of a risk label.

O.4 CSV Upload

The upload screen presents accepted format, maximum size, row limit and example columns before file selection. Drag-and-drop and file browsing lead to the same validation process. A valid preview reports row counts and resolved products; invalid input reports row, field and reason. Import is a separate deliberate action.

Upload history shows file name, date, record count, status and validation quality or error state. The interface does not imply that a failed upload changed sales. After successful commit, dashboard data can be refreshed and a completion notification links back to the upload record.

O.5 Forecasting

The forecast screen presents product, store and horizon controls. Supported horizon buttons include 7D, 14D, 28D and 30D. Running state explains that models are being evaluated. Candidate cards show model name, availability, MAE, RMSE and selection. The selected card is visually distinct and includes an explanatory label.

A line chart plots dated future quantities. Summary cards show total demand, estimated revenue and relevant stock evidence. Product-level output lists the current-to-forecast relationship and category summaries where appropriate. A saved-runs area lets the user reopen earlier evidence instead of rerunning a model simply to recover a chart.

The screen avoids presenting arbitrary accuracy percentages when only error metrics are available. Where a display score is derived, its definition is accessible. Candidate failure remains visible as unavailable rather than disappearing, making the comparison honest and helping operators diagnose Python or data sufficiency issues.

O.6 Recommendations and Insights

Recommendation cards identify product, store, type, risk, forecast link, available stock, lead-time demand, safety stock, target stock and order quantity. Primary workflow actions are Approve, Dismiss, Complete and Reopen where valid. High-risk records appear before informational records. Filters support status and recommendation type.

The Insights page groups growth, risk, opportunity and forecast interpretations. Every insight is based on calculated tenant evidence and links to the relevant product, forecast or report. It does not create unsupported causal claims. A product performance list and category breakdown help the user move from business-level patterns to individual action.

O.7 Reports

The Reports page combines key performance summaries, visual breakdowns, report-generation cards and a saved-report table. The user selects type, date range and PDF or CSV format. Generation and download actions reflect permission and status. Saved records include generated date, size, format and report type.

Forecast and replenishment reports include model and recommendation evidence, not merely a copied chart. Revenue and inventory reports use consistent units and currency. The interface identifies the reporting period so a downloaded figure can be compared with the screen without ambiguity.

O.8 Administration

The Admin dashboard presents user counts, active status, administrative roles and security events. User records show name, email, role, status, last active time and selected activity counts. Invite, edit, suspend and reactivate actions require authorization. Role and custom-permission controls explain the effect before saving.

Audit Log and System Health appear as separate tabs or views. Audit filters include action, actor and period. System activity summarizes API, database, model and notification health without exposing secrets. The interface retains the distinction between application user management and infrastructure configuration.

O.9 Profile and Settings

Profile displays user name, email, role, status and permitted business information. Edit mode allows supported personal fields such as name, phone and bio. Security-sensitive fields use dedicated forms. The danger area requires deliberate confirmation and server permission.

Settings groups Notifications, Security, AI and Models, and Appearance. Notification controls persist category and channel preferences. Security includes password and session-related controls rather than displaying a usable API key in plain text. Model settings include default horizon and preferred eligible model; the server retains automatic evidence-based selection when the preference is unavailable or inferior under policy. Appearance settings provide consistent theme selection.

O.10 Mobile Adaptation

Mobile screens preserve the task order while reducing simultaneous information. Dashboard cards stack, tables become record cards and charts receive a scroll-safe width. Header actions move into navigation or overflow controls. Touch targets remain large enough for comfortable use and forms select appropriate mobile keyboards.

Forecast candidates display vertically, with selected state and metrics readable without horizontal scrolling. Recommendation quantities are grouped into evidence and action sections. Offline or unreachable-server states provide retry and do not fabricate cached analytical results. Push notification navigation checks authentication before opening a protected target.

O.11 Accessibility and Visual Consistency

Labels remain visible independently of placeholders. Keyboard focus, semantic buttons and meaningful headings support navigation. Text contrast is maintained in dark and light modes. Risk is communicated through words and icons as well as colour. Charts have surrounding numeric summaries so essential information is not image-only.

The StockGuard brand uses dark navy, blue and teal accents consistently. The logo is displayed within its natural proportions and is not used as a background that competes with data. Spacing, card radius, table headers and status badges form one design system across pages. Mobile adapts the system instead of presenting a visually unrelated application.

CHAPTER P
PROJECT PLANNING AND MANAGEMENT

P.1 Work Breakdown

The project was divided into requirements, design, data preparation, model development, backend development, web development, mobile development, testing, deployment and documentation. Dependencies were identified early: secure identity and tenant structure preceded business data; business data preceded forecasts; forecasts preceded recommendations and reports; a stable API preceded mobile completion.

Phased delivery reduced the risk of a broad unfinished interface. Each phase ended with a runnable behaviour. The model-training notebook produced a versioned bundle. Import and verification scripts connected that bundle to the application. Subsequent phases added live Python comparison, thirty-day support, replenishment, administration, notifications and mobile parity.

Table 25: Indicative project phase plan

Phase	Principal deliverables	Acceptance evidence
Requirements	Proposal, SRS and scope	Approved documents and requirement identifiers
Architecture	MERN/Python design and tenancy	Diagrams, schemas and route plan
Data and model	Feature dataset and XGBoost bundle	Manifest, hash and recursive metrics
Core backend	Auth, products, stores, sales and uploads	Static and DB integration checks
Forecasting	Live candidates and saved horizons	MAE/RMSE and thirty-value persistence
Decision support	Replenishment, reports and insights	Formula and lifecycle checks
Web and mobile	Responsive clients	Production build, TypeScript and export
Operations	Health, performance, backup and restore	Load and recovery evidence
Documentation	User, presentation and defence reports	Rendered artifacts and traceability

P.2 Version and Configuration Control

Git records source history and allows a verified report or implementation state to be identified by commit. Dependency lockfiles preserve resolved JavaScript versions. The model manifest records project, model version, XGBoost version, training end date, features, rounds and SHA-256. Environment examples document variable names without committing credential values.

Configuration is separated by environment. Local development can point clients to a local API, while production uses HTTPS service URLs and managed secrets. Business preferences such as horizon, safety stock and notifications are stored in MongoDB because they are application data, not deployment secrets. This distinction reduces the risk of hard-coded presentation behaviour.

P.3 Change Control

A requirement change was evaluated against the submitted scope, data model, route contract, both clients and verification. Adding a 30-day horizon, for example, required backend validation, Python output, XGBoost recursion, seasonal fallback, web controls, mobile controls, settings and persistence tests. Treating it as only a new button would have created an incomplete result.

Changes that affected protected data also required permission and tenant review. Administrative features were not considered complete when only the screen existed; corresponding MongoDB routes,

audit events and status enforcement were implemented. This impact-based process helped maintain consistency as the system expanded.

P.4 Risk Monitoring

Technical risks were monitored through build checks, integration checks and production health. Data risks were monitored through validation counts and tenant fixtures. Model risks were monitored through candidate metrics and fallback. Schedule risk was managed by completing core workflows before optional integrations such as real SMTP delivery and public app-store release.

Before defence, operational risk is reduced through a stable presentation tenant, a prepared demo sequence, local copies of artifacts and a recent health check. The source repository, model bundle and encrypted backup provide separate recovery paths. Credentials remain private and are not inserted into the submitted report or source package.

CHAPTER Q
QUALITY ASSURANCE AND MAINTENANCE

Q.1 Quality Strategy

Quality assurance combines prevention, automated verification and visual review. Schema validation and service boundaries prevent predictable defects. Static checks detect missing routes, permission gaps and structural regressions. Database integration exercises real persistence, while production checks measure deployed behaviour. Web and mobile builds confirm that client code compiles in its delivery toolchain.

High-value tests focus on boundaries where an apparently working feature can fail: tenant isolation, token rotation, CSV rollback, candidate failure, exact prediction count, duplicate recommendations, authorization of downloads and isolated recovery. These checks provide more confidence than tests that repeat a simple implementation line without exercising a risk.

Q.2 Defect Classification

Table 26: Defect severity classification

Severity	Definition	Example	Response
Critical	Security breach, data loss or unusable core system	Cross-tenant access or failed recovery	Stop release and correct immediately
High	Core workflow incorrect without safe workaround	Wrong horizon count or duplicated sales	Correct before defence/release
Medium	Feature issue with controlled workaround	One candidate unavailable while fallback works	Record, prioritize and retest
Low	Minor visual or wording defect	Spacing or non-blocking label issue	Correct during polish cycle

Q.3 Regression Verification

A change to identity code triggers authentication and protected-route checks. A change to sales or products triggers upload, analytics and forecast checks because those features depend on the same records. A change to forecasting triggers candidate, persistence, revenue and recommendation checks. A change to roles triggers administration and negative-permission tests across clients.

Client changes require a production build rather than only a development server. Mobile changes require TypeScript validation and an Expo export check. Schema or index changes require testing against an isolated database. Deployment configuration changes require a production health request after release. This dependency-oriented regression policy concentrates effort on plausible consequences.

Q.4 Monitoring and Incident Response

Health monitoring checks the public service and its dependencies at a scheduled interval. A failure should be classified as web, API, database, Python-model or model-artifact readiness. Logs and correlation identifiers support diagnosis without exposing secrets. Operators first preserve evidence, then apply the smallest safe correction and repeat the affected checks.

For an authentication incident, refresh sessions can be revoked and secrets rotated. For data loss, writes should be paused before selecting a verified encrypted artifact and restoring in a controlled

environment. For model failure, the candidate can be disabled while seasonal or other live models continue. For notification failure, persistent in-app records remain the source of truth while external delivery is repaired.

Q.5 Maintenance Schedule

Table 27: Recommended maintenance schedule

Frequency	Activity
Continuous / request	Structured errors, audit events and tenant enforcement
Daily	Production health observation and encrypted MongoDB snapshot
Weekly	Review critical recommendations, failed notifications and admin activity
Monthly	Dependency audit, user access review and backup restore evidence review
Quarterly	Credential rotation review, model drift analysis and permission review
On data change	Re-run candidate metrics and inspect forecast/recommendation behaviour
On release	Build clients, run integration suites and verify production health

Q.6 Model Maintenance

Model performance should be evaluated when enough actual observations become available after a forecast. Actual-versus-predicted residuals can be grouped by model, product, store and horizon. A sustained increase in MAE or RMSE indicates drift, data change or a preprocessing problem. Retraining should create a new version rather than overwriting the current bundle without evidence.

A new model bundle must include its training range, feature order, library version, metrics and hash. Category mappings must correspond to the booster. The verification script should load the bundle on CPU, produce a smoke prediction and confirm reference forecast counts. Deployment can then update the manifest and retain the previous known-good version for rollback.

Q.7 Database Maintenance

Database maintenance includes index review, growth monitoring, invitation and token expiry, audit retention and removal of temporary verification data. Slow queries should be examined with tenant and date filters before increasing application timeouts. Indexes must balance read performance with write cost and should reflect actual query shapes.

Migration scripts should be idempotent and tested against an isolated copy. New required fields need defaults or a controlled backfill for existing documents. Destructive cleanup should follow a verified backup and explicit scope. The routine restore drill should continue to use prefixed collections so maintenance itself does not endanger production records.

Q.8 Client Maintenance

Web maintenance includes browser compatibility, responsive breakpoints, accessible focus and error-boundary behaviour. Cached assets should not prevent a user from loading a new deployment; stable recovery actions can reload the latest interface. Mobile maintenance follows supported Expo and React Native versions, secure-storage compatibility and platform permission changes.

Both clients should avoid embedding business rules that can differ over time. New screens should use the shared API contract and permission model. Analytics formatting can be client-specific, but authoritative totals, model selection and recommendation quantities remain server values. This principle reduces regression risk across platforms.

Client-side caching must respect account and tenant changes. Signing out clears protected in-memory data and a different user must not see the previous dashboard while a new request is loading. Query keys include the resource and relevant filters, while authentication state determines whether a protected request may run. Sensitive refresh values never enter ordinary browser storage or application logs.

Release review also includes navigation and recovery behaviour. Every sidebar or mobile menu destination must open a implemented screen, deep links must return through authentication safely, and an unavailable dependency must produce a readable retry state. The logo, page titles, status terminology and currency presentation should remain consistent so users can move between web and mobile without learning a second product vocabulary.

Maintenance evidence is recorded with the release rather than inferred from a running development server. The web production bundle, mobile type check and platform export are repeated after dependency or configuration changes. A short presentation smoke test then covers sign in, dashboard, product detail, thirty-day forecast, recommendation, report and sign out against the deployed API.

CHAPTER R
GLOSSARY

R.1 Terms 1 to 20

Term	Definition
Access token	Short-lived signed credential used for protected API requests.
Aggregation	Combining records into totals or grouped analytical results.
API	Application Programming Interface used by web and mobile clients.
ARIMA	Autoregressive Integrated Moving Average time-series model.
Audit log	Immutable-style record of security or business activity.
Availability	Degree to which the system can serve valid requests when needed.
Backtest	Evaluation that predicts historical future windows from earlier data.
Bcrypt	Adaptive password-hashing algorithm used for stored credentials.
Business workspace	Tenant boundary containing one organization's data and members.
Candidate model	Forecasting algorithm evaluated for a specific request.
Cold start	Condition in which little product history or a newly started runtime is available.
CSV	Comma-separated values file used for structured sales import.
Cyclical feature	Sine/cosine encoding that preserves adjacency in periodic time.
Dashboard	Summary interface for important business and operational measures.
Data leakage	Use of information during training that would be unknown at prediction time.
Demand spike	Material increase in forecast rate relative to recent history.
EAS	Expo Application Services used for React Native build and delivery workflows.
Encryption	Transformation that protects data confidentiality using a key.
Expo	Toolchain used by the StockGuard React Native application.
Feature	Numeric or categorical input supplied to a forecasting model.

R.2 Terms 21 to 40

Term	Definition
Forecast horizon	Number of future daily observations requested.
Holdout	Recent observations excluded from training and used for evaluation.
HttpOnly	Cookie property preventing browser scripts from reading its value.
Idempotency	Property that prevents a repeated operation from duplicating its effect.
Inventory coverage	Estimated number of demand days supported by available stock.
JWT	JSON Web Token carrying signed identity and session claims.
Lag	Past demand value used as a predictor for a later date.
Lead-time demand	Forecast demand expected before replenishment can arrive.
Linear Regression	Model that estimates a linear relationship between inputs and target.
MAE	Average absolute difference between actual and predicted values.
MERN	MongoDB, Express, React and Node.js application stack.
Model drift	Decline in predictive behaviour as data patterns change.
MongoDB Atlas	Managed cloud MongoDB service used for persistent data.
Multi-tenancy	Architecture serving multiple businesses with isolated records.
Notification	Persistent or externally delivered message about a domain event.
Overstock	Inventory materially above the target implied by expected demand.
Permission	Named authorization required to perform an API action.
Prediction	Estimated quantity for one specified future date.
Push token	Provider token identifying an application installation for notifications.
Random Forest	Ensemble of randomized decision trees used for regression.

R.3 Terms 41 to 59

Term	Definition
RBAC	Role-based access control assigning permissions through membership roles.
Recommendation	Saved replenishment or inventory action supported by calculated evidence.
Recovery point objective	Maximum targeted period of data that could be lost.

Term	Definition
Recovery time objective	Target duration for restoring usable service or data.
Refresh session	Revocable credential state used to obtain a new access token.
Reorder point	Stock level at which replenishment review should begin.
Reserved stock	On-hand quantity already committed and unavailable for new demand.
RMSE	Square root of mean squared error, emphasizing larger deviations.
Rolling mean	Average of a moving historical window used as a feature.
Safety stock	Inventory buffer protecting against uncertainty.
Seasonal naive	Forecast that repeats demand from a previous seasonal position.
SKU	Stock keeping unit identifying a product inside a business.
SMTP	Protocol used by a configured provider to deliver email.
Tenant isolation	Enforcement that one business cannot access another's records.
Time series	Chronologically ordered observations for one measured variable.
Token rotation	Replacement of a refresh credential while invalidating the previous value.
Target stock	Policy inventory level covering expected demand and safety buffer.
Vercel	Cloud platform hosting the StockGuard web and API deployment.
XGBoost	Gradient-boosted decision-tree library used by the production model.

CHAPTER S
DETAILED SRS REQUIREMENT SPECIFICATIONS

This appendix expands the requirement traceability matrix into testable behavioural specifications. Each subsection states the intended behaviour, major controls and completion evidence for one submitted functional requirement.

S.1 REQ-1 User Registration

The system shall allow a new user to register with a valid name, email and password and shall create an isolated business workspace for the initial owner. Email is normalized before uniqueness comparison. Password policy is validated before bcrypt hashing, and the plain password is never written to MongoDB, logs or API responses.

Successful registration creates the user, business and owner membership as one controlled workflow. A failure must not leave an ownerless workspace or unusable partial membership. Duplicate email and invalid-field cases return clear client errors. Completion is evidenced by authentication integration tests and retrieval of the newly created tenant profile.

S.2 REQ-2 Secure Login

The system shall authenticate registered active users and issue a short-lived JWT access token plus a revocable refresh session. Invalid credentials return a generic unauthorized message so the response does not reveal whether an email exists. Suspended membership or business state prevents a valid password from creating an active application session.

Web refresh credentials use an HttpOnly cookie and mobile refresh credentials use secure device storage. Refresh rotation invalidates the previous value. Logout, password reset and password change revoke applicable sessions. Completion is evidenced by valid login, invalid login, refresh, rotation, logout and revocation tests.

S.3 REQ-3 Protected Business Data

The system shall protect account and business records in transit and at rest within the controls of the selected deployment. Production communication uses HTTPS. Passwords and refresh credentials are stored as hashes, while application and workflow secrets remain in managed environment settings.

Every protected business query derives the tenant from authenticated membership and includes it in ownership filters. A client-supplied business identifier cannot widen access. Cross-tenant integration cases attempt reads and changes with a valid token from another business and confirm that no unauthorized record is returned or modified.

S.4 REQ-4 CSV Validation and Import

The system shall accept a sales CSV of no more than 10 MB and 10,000 rows and validate required headers, dates, quantities, revenue and business identifiers. Validation occurs before commit and returns row-level reasons. Unknown products or stores are resolved only within the active tenant.

Import shall avoid duplicate sales and shall not leave a partially applied batch when a blocking failure occurs. Upload history stores file metadata, counts, status and actor. Evidence includes

boundary-size tests, invalid header/date/number tests, duplicate and rollback tests, plus a successful upload notification.

S.5 REQ-5 Product Demand Forecast

The system shall generate product-store demand predictions from validated chronological sales history. Supported horizons are 7, 14, 28 and 30 days. A successful saved forecast contains exactly one prediction for every consecutive future date in the requested horizon.

The record also contains the selected model, candidate evidence, demand total, estimated revenue, requester and timestamps. Unauthorized or cross-tenant identifiers are rejected. Evidence includes complete horizon persistence and retrieval, rather than relying only on values drawn temporarily in a client chart.

S.6 REQ-6 Required Forecasting Models

The Python forecasting service shall implement Linear Regression, ARIMA and Random Forest candidates against the same prepared series and holdout. Every candidate shall return success metrics or a controlled unavailable/failure state. A failure in one candidate shall not automatically fail the entire forecast when another eligible candidate remains.

The production pipeline may also evaluate the verified global XGBoost model and seasonal-naive candidate. These additions do not replace the three algorithms committed in the proposal. Evidence includes live candidate results, Python service checks and stored candidate names and metric pairs.

S.7 REQ-7 Metric Comparison and Selection

The system shall calculate mean absolute error and root mean squared error against one common chronological holdout. It shall select the successful candidate with the lowest MAE and use RMSE as a tie-breaker. Candidate order shall be deterministic if both values remain equal within tolerance.

The selected name and all available candidate metrics shall be stored with the future predictions. The interface shall expose MAE and RMSE rather than substituting an unexplained accuracy percentage. Metric selection tests use controlled candidate outputs to verify that the correct winner is persisted.

S.8 REQ-8 Authorized Forecast Access

The system shall require a valid session, active membership, forecast permission and tenant ownership to run or read a forecast. Read and run permissions may be granted independently through roles or approved custom permissions. The server remains authoritative when a client hides an unavailable action.

Forecast lists, detail records and linked recommendations shall remain tenant-scoped. Administrative access does not imply access to an unrelated business. Evidence combines RBAC tests with cross-tenant resource attempts and confirms that protected endpoints reject an insufficient role.

S.9 REQ-9 Product and Inventory Management

Authorized users shall create, read, update and remove or archive products within their business. SKU uniqueness applies within the tenant. Product data includes name, category, price, reorder point and supplier lead time. Inventory stores on-hand and reserved quantities for a product-store pair.

Validation prevents negative prices or stock values and ownership checks prevent foreign references. Updates create audit context and become available to subsequent replenishment calculations. Product and mobile edit/delete actions use the same API. Evidence includes positive CRUD, duplicate SKU, invalid numeric and cross-tenant cases.

S.10 REQ-10 Revenue and Replenishment Insight

The system shall estimate revenue from forecast demand and current product price and shall calculate lead-time demand, historical variability, safety stock, target stock, days of cover and recommended order quantity. Formula components shall be stored or returned separately so the recommendation is explainable.

The system shall classify low stock, critical stock, overstock and demand spike, prevent duplicate open actions, and support open, approved, dismissed and completed statuses. Evidence includes unit-level formula checks, lifecycle integration and report/insight inclusion.

S.11 REQ-11 Responsive Analytics

The system shall provide dashboard analytics for products, sales, revenue, forecast activity and inventory risk. Aggregations are computed from authorized MongoDB records and respect the selected date window. Desktop and mobile layouts shall present the same underlying values in a form suited to the screen.

Loading, empty and error states shall explain system status. Charts shall be supported by readable values and units. Evidence includes tenant-isolated dashboard integration, React production build, mobile checks and comparison of prepared workspace figures with stored records.

S.12 REQ-12 Operational Alerts

The system shall create persistent notifications for upload completion, forecast readiness, low stock, critical inventory and high-risk recommendations. Each notification belongs to a business and recipient, has a type and severity, and supports unread/read state. Where metadata is available, the notification links to the relevant application record.

Email and mobile push are optional delivery channels governed by credentials and user preferences. External channel failure shall not undo the completed business event or remove the in-app notification. Evidence includes notification creation tests and controlled behaviour when SMTP is not configured.

S.13 REQ-13 Administration

Owners and authorized administrators shall invite users, list memberships, edit roles and custom permissions, suspend or reactivate access and review activity. Invitation tokens are single-use, hashed and time-limited. A suspended account shall not continue to create sessions or use protected routes.

Administration operations shall remain inside the active tenant and create audit events. The interface shall distinguish user management, audit log and system activity. Evidence includes invitation, role update, suspension, reactivation and admin authorization tests backed by MongoDB records.

S.14 REQ-14 Audit Retention

The system shall record significant identity, administration, upload, forecast, profile and security events with actor, action, tenant, target, time and sanitized metadata. Passwords, raw tokens and backup keys shall never be placed in audit metadata. Events shall be queryable by authorized users.

The implemented TTL policy retains audit records for one year, exceeding the submitted minimum of ninety days. Expiry is applied through a dedicated field and index rather than client deletion. Evidence includes event creation in integration workflows and the admin audit endpoint.

S.15 REQ-15 Role-Based Access

The system shall support owner, administrator, manager, analyst and staff role profiles with explicit permission names. Middleware shall check the permission required for each sensitive action. Approved custom overrides may refine a membership without creating a new hard-coded role.

Removing a permission shall affect later requests even when the user remains authenticated. Client visibility may improve usability but cannot replace server enforcement. Evidence includes permitted and forbidden route cases for multiple roles and a custom-permission update test.

S.16 REQ-16 Graceful Error Handling

The system shall return stable JSON errors for validation, authentication, authorization, missing resource, dependency and unexpected server conditions. Client applications shall provide recovery actions rather than an unexplained blank screen. Internal stack traces and secrets shall not be returned to users.

Model-candidate errors are isolated where possible, upload errors occur before commit and notification-channel errors do not corrupt the primary transaction. Evidence is distributed across negative cases in every integration suite, together with web error-boundary and retry behaviour.

S.17 REQ-17 Backup and Recovery

The system shall create an encrypted MongoDB snapshot every twenty-four hours through a scheduled workflow and retain artifacts for the configured period. AES-256-GCM provides confidentiality and authentication. Connection details and encryption keys are stored as encrypted workflow secrets.

A restore drill shall decrypt a controlled copy, reject tampering and restore documents to uniquely prefixed temporary collections. Collection counts and readability are verified and temporary data is removed. Recorded runs restored sixteen collections in seconds, within the ten-minute recovery objective.

S.18 REQ-18 Privacy and Secure Communication

The system shall combine HTTPS, password hashing, signed sessions, revocation, role permissions and tenant filters to protect user and business data. Clients shall never receive database credentials or private model/backup secrets. Mobile refresh credentials shall use secure device storage.

Privacy also requires data minimization. Logs and notifications include only the context required for operation. The application makes forecasts advisory and retains a human decision for

replenishment. Evidence includes production HTTPS, secret-free source packaging, tenant isolation and security integration checks.

CHAPTER T
WORKED BUSINESS SCENARIO

T.1 Scenario Context

TechMart Pakistan is used as an illustrative SME workspace for the internal defence. It sells electronics, sports products, food and fashion items through multiple stores. The business owner wants to reduce urgent stockouts without tying excessive cash in slow-moving products. Team members include an owner, an administrator, analysts and viewers whose access differs by responsibility.

The prepared workspace contains eight representative products and eight hundred sales records. It also contains a complete 30-day saved forecast, reports, notifications and replenishment recommendations. The scenario is not presented as a live commercial customer; it is a controlled demonstration tenant designed to exercise the final application without modifying unrelated production data.

T.2 Product and Inventory Setup

The catalogue includes products such as Samsung Galaxy S24, Nike Air Max, Organic Rice, HP Laptop, Levi's Jeans and Sony headphones. Each item has a SKU, category, selling price, reorder point and supplier context. Store-level inventory records separate on-hand and reserved units. Supplier lead time is configured per product or inventory policy because electronics and food may have different replenishment delays.

Before forecasting, the manager checks that product and store identifiers in the sales export correspond to the catalogue. The CSV validation preview confirms required fields and flags any unknown SKU. This step is operationally important: a forecast attached to an incorrect product may be statistically valid for the wrong series and therefore useless for purchasing.

T.3 Historical Sales Import

The manager selects the prepared CSV and runs validation. The parser confirms the file size and row count, checks headers and converts date, quantity and revenue fields. Duplicate product-store-date keys are handled under the import policy. Because the demonstration file is valid, the preview reports eight hundred accepted sales records and no blocking error.

On commit, the batch is written inside the TechMart tenant and an Upload record stores file name, counts, actor and completed status. The dashboard updates from database aggregation, and an upload-completed notification appears for the relevant user. Re-uploading the same business keys does not double the analytical quantity.

T.4 Thirty-Day Forecast

The analyst opens AI Forecast, chooses one product-store series and selects 30D. The server loads only TechMart sales for that series and creates the continuous daily history. It evaluates the live Linear Regression, ARIMA and Random Forest candidates. When trained mappings apply, XGBoost is added; seasonal naive remains available for comparison and resilience.

Candidate cards show MAE and RMSE or a controlled unavailable state. The selector chooses the lowest MAE and applies the tie-break rule. The stored result contains thirty consecutive future dates,

quantities, total demand and estimated revenue. The analyst can reopen the same saved record from the web or mobile application without running the models again.

The daily sequence is important for this scenario because the supplier lead time is shorter than the full horizon. The replenishment service sums only predictions expected before arrival for lead-time demand, while the remaining days contribute to target coverage according to the review policy.

T.5 Replenishment Decision

For a low-stock product, available inventory is calculated after reserved units are removed. Historical variability and lead time determine safety stock. Lead-time and review-period predictions determine the demand component of target stock. The resulting order quantity is shown with all components, risk severity and days of cover.

The manager checks whether a promotion, supplier delay or already placed external order changes the interpretation. If the evidence remains appropriate, the recommendation is approved. If an unrecorded fact makes it unsuitable, it is dismissed with the history retained. When the order has been placed or received, the action can be marked completed. A repeated forecast does not create multiple open copies of the same risk.

T.6 Reporting and Insight

The manager generates an inventory or forecast report for the selected period. The report uses the saved forecast and recommendation rather than recalculating a different client-only total. Revenue, model identity, error metrics, daily predictions and replenishment quantities can be inspected together. The saved report remains in the TechMart workspace and can be downloaded only by an authorized member.

The Insights page explains that the product is exposed to stockout risk or that a category is expected to grow. The explanation links to the underlying product or forecast. It does not claim that the model has discovered the cause of growth. The owner can combine this quantitative evidence with current supplier and promotion information.

T.7 Mobile Review

Away from the desktop, the owner signs in through the Expo mobile application. SecureStore maintains the refresh session and the same API returns TechMart data. Dashboard, forecast and recommendation screens reorganize the information into stacked cards. The thirty daily predictions and selected model remain the same because the mobile client does not recalculate them.

A critical notification can open the relevant recommendation when the session is valid. The owner reviews the quantity and status and can perform an authorized workflow action. Audit history attributes the change to that user regardless of client platform.

T.8 Administrative Review

The owner opens Admin and reviews active users, roles and recent security events. An analyst has forecast permissions but cannot suspend users. A viewer can inspect allowed reports but cannot change inventory. If a team member leaves the business, suspension blocks later access while historical audit attribution remains intact.

Settings confirm the default thirty-day horizon, notification preferences and appearance. A password change verifies the current credential and revokes older sessions. The owner can inspect health information without receiving database or backup secrets. These actions demonstrate that governance is implemented behind the presentation rather than represented by static controls.

T.9 Operational Evidence

The defence concludes by showing model backtest, performance and recovery evidence. The global XGBoost model's combined recursive metrics are compared with seasonal naive. The deployed load test shows one hundred successful authenticated responses with p95 below the requirement. The restore evidence shows that encrypted data can be decrypted, imported into temporary collections, verified and removed.

Together, the scenario demonstrates the intended value chain: controlled identity, valid data, comparable forecasts, complete persistence, inventory interpretation, human approval, cross-platform access and recoverable records. No single screenshot establishes this chain; the connected workflow and evidence do.

CHAPTER U
EVIDENCE REGISTER

U.1 Model Evidence

Table 28: Verified production model evidence

Evidence item	Recorded value	Interpretation
Training rows	8,172,438	Large global feature matrix after preparation
Boosting rounds	83	Early-stopped production tree ensemble
Feature count	12	Identifiers, lags, rolling and calendar features
Backtest observations	986,433	Combined three-fold recursive evaluation
XGBoost MAE	0.574939	Primary combined absolute error
XGBoost RMSE	2.021802	Combined squared-error-sensitive measure
Seasonal MAE	0.658729	Baseline absolute error
Seasonal RMSE	2.459768	Baseline squared-error-sensitive measure
MAE improvement	12.72%	Relative improvement against seasonal baseline
Forecastable series	11,626	Series included in production forecast bundle
Bundle size	3.44 MB	Portable model and supporting artifacts

U.2 Application Performance Evidence

Table 29: Verified performance measurements

Measurement	Result	Requirement interpretation
Authenticated virtual users	100	Target concurrency exercised
Successful responses	100/100 HTTP 200	No error during measured ramp
Average response	259 ms	Common warm request performance
p95 response	832 ms	Below three-second interactive target
Maximum response	1,210 ms	Below three seconds in measured run
Login	855 ms	Below submitted login target
Dashboard	843 ms	Below five-second dashboard target
Thirty-day forecast	2,224 ms	Below ten-second forecast target
10,000-row validation	61 ms	Local validation path within practical limit

U.3 Recovery Evidence

The machine-readable local drill restored 1,055 documents across sixteen collections in 13.914 seconds. A later scheduled production run restored 1,137 documents across sixteen collections in 23.454 seconds. The difference in document count reflects continued workspace activity, while both durations remain far below the ten-minute recovery objective.

Evidence also confirms removal of prefixed temporary collections after verification. The report distinguishes the implemented daily encrypted snapshot from optional Atlas point-in-time recovery. This avoids claiming a cloud-plan feature that is not required for the verified repository workflow.

U.4 Artifact Integrity Evidence

The model manifest records SHA-256 value 61dda9da18aa090ed26304319c7162215f0a0158a46af18dbfcc555f92851c3c for the imported XGBoost JSON. The local verification script loaded XGBoost 3.2.0, confirmed twelve features, 11,626 forecast series, 325,528 daily reference rows and a 28-day reference date range from 1 to 28 November 2019.

The hash establishes byte-level identity of the verified model artifact. It does not by itself establish predictive quality, which is supported separately by recursive backtesting. Maintaining both types of evidence prevents integrity and accuracy from being confused.

U.5 Evidence Limitations

Performance results describe a bounded run and do not replace long-term uptime measurement. Recovery results demonstrate archive usability and timing in the tested workflow but do not simulate every disaster condition. Model metrics reflect the selected retail dataset and validation windows, not every future SME series.

The appropriate conclusion is that the prototype satisfies its submitted verification targets with reproducible evidence. Continued production use should accumulate monitoring history, actual-versus-forecast residuals, user feedback and recommendation outcomes.

CHAPTER V
INTERNAL DEFENCE QUESTIONS AND EVIDENCE

The following questions summarize the technical reasoning most likely to be examined during the internal defence. Answers identify the evidence that should be demonstrated rather than memorized as unsupported claims.

V.1 Why were several forecasting models implemented?

Retail series differ in trend, seasonality, volume and intermittency, so one model is not consistently best. StockGuard evaluates Linear Regression, ARIMA and Random Forest on the same chronological holdout and can also evaluate eligible XGBoost and seasonal candidates. The saved MAE and RMSE values show why a model was selected for the particular series.

V.2 Why is MAE the primary selection metric?

MAE expresses average deviation in the same units as demand and is stable when actual quantity is zero. RMSE remains useful because it penalizes large misses more strongly, so it is reported and used to resolve a tie. Percentage error was not selected because zero-demand dates can make it undefined or misleading.

V.3 How is data leakage prevented?

Sales are ordered chronologically and the most recent observations form the holdout. Lag and rolling features use only values available before the target date. During recursive evaluation, earlier predictions replace unknown future actual values. Scaling or fitting is performed on training data before evaluation. No random split allows future behaviour to enter the past.

V.4 What makes the thirty-day requirement complete?

The horizon is accepted by backend validation, Python candidates, XGBoost recursion, seasonal fallback, web controls, mobile controls and business settings. An integration check retrieves the saved forecast and confirms exactly thirty unique consecutive daily predictions, candidate metrics, demand total and revenue. A button alone would not satisfy the requirement.

V.5 How is a forecast converted into an order quantity?

The service calculates available inventory after reservations, sums demand inside supplier lead time, estimates historical variability and safety stock, adds review-period demand to produce target stock, and subtracts available stock. The non-negative gap is the recommended quantity. Every component is shown so a manager can challenge an assumption before approval.

V.6 How is business data isolated?

The authenticated membership supplies the business identifier. Products, stores, sales, uploads, forecasts, recommendations, reports, notifications and audit events are queried with that tenant key. Permission middleware controls actions, and resource ownership is checked again for specific identifiers. Cross-tenant integration tests use a valid foreign token and confirm denial.

V.7 What happens if the Python service is unavailable?

The API treats model results as independent candidates. A timeout or fitting error marks the affected candidate unavailable without exposing an internal stack trace. Eligible XGBoost or seasonal-naive forecasting can still provide a result under policy. The candidate evidence records the fallback truthfully, and health output identifies reduced Python readiness.

V.8 What evidence shows production readiness?

Evidence includes successful web and mobile builds, authenticated database integration, thirty-day persistence, role and tenant tests, a one-hundred-user deployed load test with zero errors, model artifact hash verification, scheduled health monitoring and an encrypted isolated restore of sixteen collections. These results support the defined prototype targets while long-term monitoring remains future operational evidence.